Formal Verification of the Lightning Network in Tamarin

***Abstract*—The Lightning Network routes payments over chains of payment channels using hash time-locked contracts (HTLCs). One Tamarin theory models its channel layer (unboundedly many updates, revocation, punishment) and multi-hop routing over routes of any length, against a Dolev-Yao adversary that can compromise long-term keys. We prove that only cheating or compromised parties are punished, that the receiver's invoice authenticates settlements, and that preimages, which our model never sends, stay secret. Under three liveness restrictions, which prune violating runs, an uncompromised intermediary never has its outgoing HTLC redeemed while its incoming one is refunded, and payment atomicity holds without key compromise. Timely claiming is necessary: without it, an intermediary can lose funds in a compromise-free run. In a Tamarin-checked wormhole trace, colluders bypass an honest intermediary, which is refunded but earns no fee. For routes of any length, a hop of a settled payment whose hash is offered once is bypassed only if some long-term key is compromised. On a fixed route, per-hop locks as in point time-locked contracts prevent bypassing an honest intermediary when the sender is honest; plain HTLCs admit it. All results are machine-checked with Tamarin 1.10.**

***Index Terms*—Lightning Network, payment channels, HTLC, wormhole attack, symbolic verification, Tamarin**

I. Introduction

a) Payment Channel Networks: Bitcoin [1], the leading cryptocurrency by market capitalisation, is costly, has low bandwidth (a few dozen transactions per second) and high latency. Payment channels let two users pool funds, then perform arbitrarily many payments quickly and cheaply via balance updates, relying on the blockchain only to resolve disputes; revocation ensures that no honest party loses money. As a channel per pair of users is often not cost effective, payment channel networks route funds transitively across channels. The most popular is perhaps the Lightning Network [2], with about $170M locked [3]. Due to the sheer complexity of its specification [4], it suffers from many attacks [5], [6], [7]. Despite promising ongoing efforts [8], [9], it still lacks a machine-checked model of both the channel layer and multi-hop routing for unboundedly many parties, channels, updates and payments and for routes of any length ([8] model-checks finite instances in TLA+). This paper attempts to alleviate that issue.

b) Protocol Verification: Designing and analysing cryptographic protocols is notoriously difficult and error-prone, as the attacks and tentative fixes on the Lightning Network illustrate. In the computational model, inherited from cryptography, attackers and participants are polynomial-time algorithms and results are typically probabilistic; its tools, such as CryptoVerif [11], EasyCrypt [12], Squirrel [13] and CryptoVampire [14], are all brittle, hard to use or narrow in scope. The symbolic (*Dolev-Yao*) model [10] gives the attacker infinite computational power but limits it to a fixed term algebra, abstracting away from cryptography to focus on the logic of a protocol, where much attacker activity is found; it enjoys successful automated verifiers, ProVerif [15] and Tamarin [16]. Mechanised proofs cover widely used protocols such as WireGuard [17] and TLS 1.3 [18], the latter in Tamarin, which proves trace properties for unboundedly many sessions and states assumptions explicitly, as trace restrictions.

c) Contributions: All results are checked with Tamarin 1.10 and reproduced by `make all` (Sec. V-B); all-traces statements hold for every trace satisfying the model's restrictions (Sec. IV).

- *One theory, one adversary.* `lightning.spthy` covers the channel layer (opening, unboundedly many updates, revocation, cooperative and unilateral closing, punishment) and multi-hop HTLC routing, with one generic rule, `Forward_HTLC`, for every hop of routes of any length. The Dolev-Yao adversary can also compromise any long-term key at any time, and two compromised parties on a payment's route can relay its preimage around the parties between them (`AttackerSkip`; Sec. IV-A). The layers share the channel topology and a per-channel slot; their contract, that each HTLC is resolved at most once, is a named restriction (`OneResolutionPerHTLC`) justified informally, not a composition theorem (Sec. IV).
- *Safety and liveness.* Among 32 all-traces safety lemmas: a party is punished only if it published a revoked state or its key was compromised; a sender settles only against the receiver's invoice, unless the receiver's key was compromised; preimages stay secret (our model never sends them); no HTLC is both redeemed and refunded. Under three liveness restrictions, T1 to T3, an intermediary whose outgoing HTLC is redeemed has its incoming one refunded only if its key was compromised before that HTLC timed out, and payment atomicity holds without key compromise. The restrictions remove every violating trace, including traces in which an uncompromised forwarder could not have acted in time; our results say nothing about such traces. Ten exists-trace lemmas show that the protocol runs and that attacks with compromised keys are reachable (Sec. V).

T3 (`IntermediaryMustClaim`) is necessary: without it, TAMARIN finds a compromise-free trace in which an intermediary's outgoing HTLC is redeemed while its incoming one is refunded. None of the five liveness lemmas we check needs T1, and neither intermediary-safety lemma needs T2, from which the CLTV-delta ordering of timeouts (`ForwardTimeGap`) is derived (Sec. V-D).

- *The wormhole: trace and theorem.* TAMARIN checks (by replaying a saved proof) a five-party trace where the payment settles while an honest intermediary is refunded on its outgoing HTLC, earning no fee (Sec. V-E). Conversely, if the sender settles, a forwarder on its route is refunded on its outgoing HTLC only if some long-term key is compromised: at each of the first six hops of a route of any length under the liveness restrictions and, if the payment hash is offered once, at every hop without them (so also with them). Neither result says whose key is compromised, or when (Sec. VI).
- *Countermeasure.* On a fixed route of three intermediaries, per-hop locks derived with secret tweaks (a symbolic stand-in for point time-locked contracts, PTLCs [6]) prevent bypassing the honest middle hop unless the sender is compromised; plain HTLCs admit it (Sec. VI-E).

Sec. II recalls the protocol, Secs. III–VI give our model and results, Secs. VII–VIII related work and limitations.

II. PROTOCOL OVERVIEW AND THREAT MODEL

The Lightning Network has two main parts. The *payment channels* transfer funds quickly and efficiently between two participants with reduced interaction with a blockchain, after a comparatively expensive setup; the *routing* protocol scales these pairwise payments to a whole network. In the channel, revocation keys protect against *outdated states*; in routing, a payment hash and a deadline control the *transfer*. They meet only in the commitment transaction, where an in-flight payment is an extra output subject to the same revocation as the rest of the channel.

A. Payment Channels

A payment channel lets two agents *commit* and *revoke* a shared "state" tracking the balance of a multi-sig [19] *common fund*, effectively transferring coins between them. It is represented by valid transactions spending that fund. Careful signing (restricting transactions to agreed-upon shapes) and *punishing* ensure that no agent can publish an old, favourable state: a participant may unconditionally retrieve *all* funds in the channel if its counterparty puts an old state on chain. In the Lightning Network, the state also tracks in-flight payments propagating beyond the channel, carried by *hash time-locked contracts* (HTLCs; Sec. II-B).

1) The Transactions: Fig. 2 shows the commitment transaction held by Alice in an Alice-Bob channel, which enables unilateral closing; our model abstracts it, but its shape is worth keeping in mind. Outputs 3 and 4 redistribute the common fund between Alice and Bob; Outputs 1 and 2 lock the funds of in-flight payments. Unlike Bob's

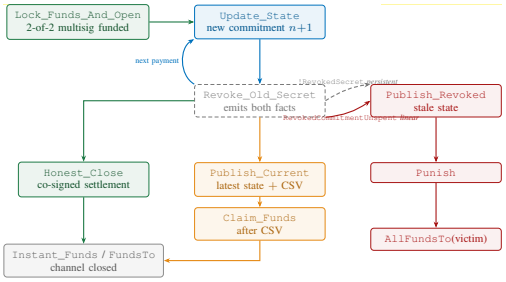


Fig. 1. Channel lifecycle, labelled with the rules of our model. Green: opening, cooperative close; blue: update loop; orange: unilateral close (CSV delay); red: revoked publish, punishment. Both facts emitted by `Revoke_Old_Secret` gate punishment: the persistent `!RevokedSecret` (dashed, never consumed) and the linear `RevokedCommitmentUnspent` (solid, consumed once; Sec. V-B). Closes end in settlement, punishment in `AllFundsTo`. Grace-period and pending-publish paths and the counterparty's instant output are elided.

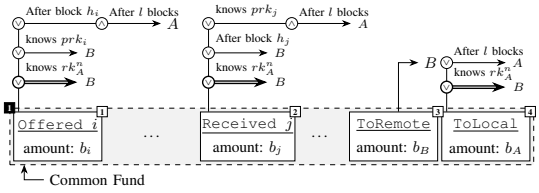


Fig. 2. Commitment transaction held by A: CSV delay l; CLTV timeout h_k and preimage prk_k of the kth HTLC (`Offered` and `Received` outputs); ⇒: punishment path; 1: an output.

Output 3, Output 4 is redeemable by Alice only after a relative timelock (CSV, for the `CheckSequenceVerify` opcode), which gives Bob the opportunity to redeem it with Alice's *revocation key* (rk_A), initially hidden from Bob. Output 1 (resp. 2) holds in-flight funds flowing towards Bob (resp. Alice). Besides punishment, these outputs are governed by a *payment hash* and a timeout: to *fulfil* the transfer, the receiver (Bob for Output 1) needs the preimage of the payment hash; to prevent funds from being locked up, the output reverts to a regular output at block height h_i, an absolute timeout set with CLTV (the `CheckLockTimeVerify` opcode). The real Lightning Network separates the CLTV and CSV conditions with a two-stage construction, which Fig. 2 collapses into a single output per HTLC. Our model keeps counterparts of Outputs 3 and 4 and of the revocation path only; HTLCs are resolved in the routing layer (Sec. IV).

2) The Protocol: The lifecycle of a channel has five phases, which correspond, left to right, to Fig. 1.

a) Opening: To prevent griefing attacks, the opening establishes a first committed state before the common fund. After an unmodelled negotiation of the inputs of the common fund, Alice and Bob each sample a random revocation key (rk^0_A and rk^0_B) and exchange their hashes. Relying on SegWit [20], both compute the TXID of the common fund, an on-chain output locked by a 2-of-2 multisig [19], and exchange the signatures needed to spend it (e.g., Bob signs the outputs of Fig. 2; Sec. V-A1 describes

our abstraction). The witnesses for the common fund are then exchanged and the funding transaction is published; once it is on chain, the channel is ready.

b) Update: The parties pay each other off chain by exchanging signed commitment transactions. They first commit to a new state n as in the opening (sampling rk_A^n and rk_B^n, exchanging their hashes and the signatures), and only then revoke state $n-1$ by disclosing its revocation keys (e.g., A sends rk_A^{n-1} to B). Reversed, these steps would leave a party nothing to publish if its counterparty stopped responding (old state revoked, new one unsigned).

c) Revocation: Alice retires state n by giving rk_A^n to Bob, proving that she will not use it: the key is useless unless Alice publishes that state. A commitment not yet revoked is a valid unilateral close, publishable without penalty; only publishing a revoked one triggers punishment.

d) Closing: A *cooperative* close is one transaction, signed by both parties, that sends the agreed final balance directly to their wallets: no scripts and no delay, the cheapest outcome for both. In a *unilateral* close, either party publishes its current commitment alone; the CSV delay on its own Output 4 is a punishment window in which the counterparty can check whether the state was revoked and, if so, take that output outright. Unilateral closing thus disadvantages the publisher, and an honest party uses it only when its counterparty has stopped responding.

e) Punishment: If a party publishes an old state, whose revocation key its counterparty knows, the latter can take the whole channel balance via the punishment paths of Fig. 2. Punishment is an *active* defence: the defender must notice the fraud, send the punishment transaction and have it accepted within the CSV window. Failure to react makes the protocol trivially insecure [21].

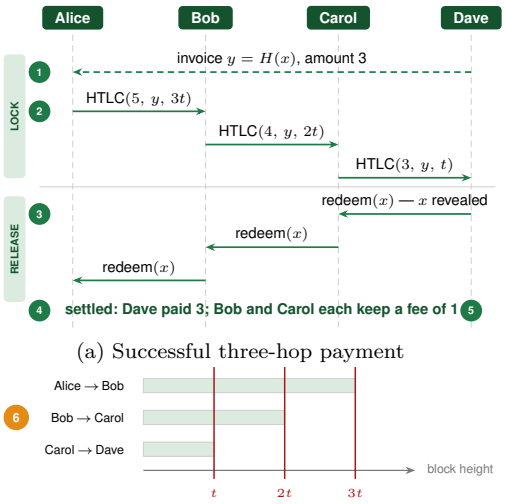
B. Multi-hop HTLC Routing

When the sender and recipient do *not* share a channel, the payment is routed through intermediaries as in Fig. 3: it is a *multi-hop* payment. As intermediaries are untrusted, each hop relays the payment in an HTLC conditioned on a single shared secret, the *payment hash*, so that the payment either completes along the entire path or fails; the wormhole attack (below) breaks this for the hops between colluding intermediaries. Intermediaries earn a routing fee.

Example 1 (Path)**.** *We will use the running example where S sends money to R with the following path:*

$$S \to F_1 \to \cdots \to F_n \to R \tag{1}$$

a) Locking: The receiver (R in Ex. 1) generates a preimage x, keeps it strictly secret, and sends the sender (S) an invoice with the hash $y = H(x)$ and the requested amount v. The sender adds an HTLC to its channel with F_1, locking funds v_0 that a preimage x' with $H(x') = y$ unlocks before the deadline d_0 (as in Fig. 2); after d_0, the payer may reclaim them. Each forwarder in turn locks



(a) Successful three-hop payment

(b) Griefing

Fig. 3. In-flight payments on the route Alice→Bob→Carol→Dave. (a) Amounts ($5 \to 4 \to 3$, one-unit fees) and deadlines ($3t > 2t > t$) decrease along the route. (b) Dave never releases x: every HTLC runs to its deadline and the payment fails, but the nested deadlines refund every honest party, so griefing locks capital (denial of service) without loss of funds. Circled numbers index the lemma groups of Table IV.

funds under the same hash, with decreasing funds and deadlines, the routing fee $v_{k-1} - v_k$ going to F_k:

$$v_0 > v_1 > \cdots > v_n = v \tag{2}$$

$$d_0 > d_1 > \cdots > d_n \tag{3}$$

The funds have not moved yet: each HTLC is an additional conditional output in the two parties' current commitment transaction. F_i can safely lock funds, since its incoming HTLC has a larger amount and a later deadline: if the outgoing HTLC is claimed, F_i learns x and still has time to unlock the incoming funds, keeping the difference as fee (Sec. V-A4 gives our abstraction of amounts and deadlines).

b) Release Round: The receiver reveals x to the last forwarder to claim the final HTLC. Since every HTLC has the same preimage, each intermediary learns x when the next hop claims from it and uses it to claim its incoming HTLC, keeping its fee: x propagates backward hop by hop until the sender's HTLC is settled. If a party fails to forward it and the timelock expires, the funds return to the party that locked them. Once x is released, all HTLCs are settled if every intermediary claims its incoming HTLC in time; one that fails to do so after its outgoing HTLC was redeemed loses the forwarded amount, while the parties downstream are paid. Hence forwarders need liveness (Remark 2).

Remark 1 (Claiming and Liveness)**.** *Claiming can happen by mutual negotiation or by unilateral closing. Notably, unilateral closing also publishes x, as it is leaked by the closing transactions redeeming the HTLC.*

Remark 2 (Decreasing Deadlines and Liveness)**.** *Consider F_i holding an incoming HTLC with deadline d_{i-1} and an outgoing one with deadline d_i. If F_{i+1} unilaterally claims*

the outgoing HTLC at d_i, F_i learns x as late as d_i and is left the window (d_i, d_{i-1}) to settle its incoming one. This gap, the CLTV delta, is precisely the time reserved for F_i to recover the funds: in case of unilateral close, forwarders need liveness to claim their incoming HTLC in this window [21].

Our model imposes this liveness as restriction T3 (Sec. IV-B), without which intermediary safety fails (Sec. V-D). Finally, as all HTLCs of a payment share the hash y, two colluding intermediaries can pass x around the hops between them. In this wormhole attack [6], the payment completes while the bypassed hops are refunded and earn no fee (Secs. V-E and VI).

III. Symbolic Model and Notation

We model the protocol using Tamarin [16], a symbolic tool for security-protocol analysis. It tracks a *system*, a multiset of facts called the state S, which evolves following user-defined transition *rules* $[L] \text{—}[A]\rightarrow [R]$: when the premises L unify with facts in S, they are rewritten as the conclusions R, and the action facts A are appended to the trace. Linear facts are consumed upon rule application; persistent facts (prefix `!`) can be reused arbitrarily often. The built-in facts `In`, `Out` and `Fr` capture receiving from and sending to a public (i.e., attacker-controlled) channel, and generating fresh, unique values. We use the hashing (h, for payment hashes $y = h(x)$ and the commitment $h(s)$ to a revocation secret s), signing (`sign`, `verify` and `pk`, for the channel handshake, the commitments and the invoices) and natural-numbers built-ins (amounts, prefixed with `%`, with addition, for the per-hop fee constraint of Sec. V-A4). Security goals are first-order formulas over traces of action facts (with time points `#i`): `all-traces` lemmas are security queries over all traces, `exists-trace` lemmas reachability queries. A *restriction* is a formula of the same kind that Tamarin imposes on the model: only traces satisfying every restriction are considered.

IV. Formal Modelling Methodology

We model the protocol as a single Tamarin theory, `lightning.spthy` (37 rules), with one adversary. It contains both the payment-channel protocol (Sec. II-A) and multi-hop routing (Sec. II-B); one generic rule, `Forward_HTLC`, handles every hop, so routes of any length are covered. The routing layer does not abstract the channel layer away, but the coupling is deliberately thin: commitment transactions carry no HTLC outputs, so the channel state is independent of the payments, and the layers share only the long-term keys and three explicit interfaces.

a) Topology and Slot: `Lock_Funds_And_Open` mints the persistent facts `!ChannelConnect`, one per direction, which `Sender_Offer_HTLC`, `Forward_HTLC` and `Open_Timeout` require, so HTLCs are added only on opened channels (`HTLC_On_Opened_Channel`). It also mints one linear fact `Mutex(ptr)`, which adding an HTLC consumes and redeeming or refunding that HTLC returns: a channel holds at most one in-flight HTLC at a time and can be reused afterwards.

b) Resolution: Every rule that redeems or refunds an HTLC records the action fact `Old`, and the restriction `OneResolutionPerHTLC` (Appendix B-A) lets each HTLC be resolved at most once. This interface assumption holds in the protocol because an HTLC output of the current commitment can be spent only once on chain, and spending it from an older state requires publishing a revoked commitment, which the counterparty can punish (`Publish_Revoked_A/B`, `Punish_A/B`). The argument is not mechanised, and we claim no composition theorem; a mechanised discharge (by composition, or by refinement as in [8]) is the natural next step. All other abstractions are also explicit, named restrictions (Sec. IV-B).

A. Threat Model

Our adversary is the standard Dolev-Yao adversary [10] of Tamarin: it controls the whole network and can read, modify and inject messages of arbitrary complexity within the term algebra. Public keys, channel proposals and acknowledgements, and invoices are sent over the network. It also chooses the values honest rules read from it: the balances of a channel update (`Update_State`) and each hop's incoming amount, fee and outgoing amount (`Forward_HTLC`), subject only to the incoming amount being the outgoing amount plus the fee. HTLC offers and fulfilments between adjacent hops are linear facts (`Route`, `Fulfilled`), i.e., authenticated, non-replayable messages between channel partners.

a) Key Compromise: Rule `Compromise_Ltk` reveals the long-term key of any party at any time (`LtkCompromised`); the adversary can then sign as that party, e.g., forge its invoices, so that a payment without an invoice of the receiver can settle (`Forged_Invoice_Requires_Key_Compromise`). The preimage x is never sent on the network, and the adversary never learns it (`Preimage_Never_Known`); instead, two compromised parties with HTLCs on the same payment hash can relay x around the parties between them with the rule `AttackerSkip` (Secs. V-A3 and V-E): the wormhole attack [6]. Apart from their keys, `AttackerSkip` and the exemptions of Sec. IV-B, compromised parties follow the protocol rules.

b) Time and Liveness: Time is abstract: the timeout of channel `ptr` is the event `TimedOut(ptr)` of rule `Open_Timeout`, which can happen at any point after the channel opens, at most once per channel (`OneTimeoutPerPtr`), so the adversary chooses when timeouts happen relative to honest actions. Controlling the network, it could also keep a party from noticing fraud or sending the defending transaction, which trivially breaks punishment (Sec. II-A2e) and claiming (Remark 1) [21].

B. Liveness Restrictions

We therefore impose liveness on honest forwarders by three restrictions on the routing layer, which limit the

adversary's control over the order of events and rule out pathological honest behaviour (the flags `NO_RBT`, `NO_OTF`, `NO_IMC` remove them for the necessity runs):

```
restriction RedeemBeforeTimeout:
    "All ptr y #i #j.
        Redeemed(ptr, y) @ #i & TimedOut(ptr) @ #j ==> #i < #j"
restriction OutgoingTimesOutFirst:
    "All P Prev Next ptrIn ptrOut y #f #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrIn) @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #m. TimedOut(ptrOut) @ #m & #m < #k"
restriction IntermediaryMustClaim:
    "All P Prev Next ptrIn ptrOut y #f #i #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & Redeemed(ptrOut, y) @ #i
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #l. Redeemed(ptrIn, y) @ #l"
```

T1 (`RedeemBeforeTimeout`): *if an HTLC is redeemed and its channel times out, the redemption comes first;* a well-formedness condition, not a liveness assumption. **T2** (`OutgoingTimesOutFirst`) is the decreasing-deadline (CLTV-delta) invariant of Remark 2: *if the incoming HTLC of a hop times out and the forwarder was not compromised before, its outgoing HTLC timed out earlier.* **T3** (`IntermediaryMustClaim`) is the claiming liveness of Remarks 1 and 2, on which intermediary safety rests: *if a forwarder's outgoing HTLC is redeemed and its incoming HTLC times out, and the forwarder was not compromised before that timeout, the incoming one is redeemed too.* It only constrains traces with an incoming timeout, the only ones that admit an upstream refund (`Refund_Requires_Timeout`); by T1, the claim then precedes the timeout.

Tamarin derives the order of a hop's two timeouts from T2 and `OneTimeoutPerPtr` (`ForwardTimeGap`, Appendix A-H): *if both channels of a hop time out and its forwarder is not compromised before the incoming timeout, the outgoing one times out strictly first.* As restrictions, T1 to T3 remove every trace that violates them, including traces in which an uncompromised forwarder could not have acted in time; our results say nothing about such traces (Sec. V-D). The liveness of punishment (Sec. II-A2e) is not encoded: no restriction mentions cheating or punishment. Only T3 is necessary for intermediary safety (Sec. V-D). The bypass theorem for routes of any length and a single offer per payment hash (Sec. VI-B) is proved without T1 to T3, all other results of the main theory under all nine restrictions (six structural; Appendix B-A).

C. Structural Limitations

Verifying the Lightning Network as one monolithic block hits the scalability limits of verification tools [8], Tamarin included: with signatures, an equational theory, natural-number induction and an unbounded clock, the search space explodes and Tamarin exhausts memory; without the unbounded clock, the others are tractable together. The main theory therefore represents timeouts as untimed events ordered by the restrictions above. Two of the five small, self-contained models of `lightning_aux.spthy` (hashing and natural numbers only, not composed with the main theory; Table I) analyse the block clock and the CLTV arithmetic; the others are the PTLC countermeasure, its HTLC baseline and the early-timeout race.

Five deliberate non-goals buy generality or tractability. *Onion privacy:* any channel partner may be the next hop and nothing is encrypted in layers, so the results cover every route but claim no privacy. *On-chain fees:* publishing a transaction is free, separating protocol logic from fee economics. *Fee policy:* the adversary chooses every routing fee, so the results hold for any policy. *Wallet capacity:* HTLC amounts are not checked against balances, so the results hold for all amounts. *Per-HTLC CSV stage:* HTLCs are resolved in one step, with no second-stage transaction.

V. Verification of the Model

We describe the model (Sec. V-A) and what we verified (Sec. V-B). Listings are verbatim; Appendix B prints further rules.

A. The Core Model

1) Payment Channels: A channel is identified by a fresh pointer `ptr` to its founding transaction. Our model tracks two mirror facts about this channel, `StateA` and `StateB`, one held by each party, each carrying the two balances (`fA`, `fB`), a state index `n`, and that party's own revocation secret together with the corresponding commitment. The commitment is stored without its signature; signatures are checked when a state is installed (`Both_States_Committed` for the first, `Revoke_Old_Secret` for every later one) as `Eq` action facts; a rule cannot fail, so restriction `Equality` enforces them.

The heart of the model is the *update dance* (Appendix B-B): commit a new state first, then revoke the old one, so that the old state is never revoked until a fully signed new one exists. `Update_State` samples the next state index and both fresh revocation secrets, reads the new balances from the environment, and turns the two mirrors into two pending mirrors that keep the old state next to the new, signed one. Only afterwards does `Revoke_Old_Secret` install the new state, recording the old revocation secrets as persistent knowledge (`!RevokedSecret`) and the old commitments as linear one-shot tokens (`RevokedCommitmentUnspent`, Sec. V-B).

`Lock_Funds_And_Open` mints the two mirrors, the persistent facts `!ValidOnChain(ptr)`, `!ChannelRecord(ptr, $A,$B)` and `!ChannelConnect` (one per direction), and the per-channel slot `Mutex(ptr)`. The remaining phases reuse these facts (Appendix B-C). A unilateral close spends the current mirror: `Publish_Current_A` turns `StateA` into an `OnChainDelayed` and an `OnChainInstant` output, and `Claim_Funds_A` later releases the delayed output to its owner; its action `CSVPassed` captures that time has passed. A cooperative close (`Honest_Close`) signs one shared settlement transaction instead, and punishment (`Publish_Revoked_A`, then `Punish_A`) transfers the whole channel to the honest party.

2) Routing: Each direction of the routing layer (Sec. II-B) has its own fact. HTLC offers travel forward as linear facts `Route($Prev,$Me,ptrIn,y)`, each also recorded in a persistent fact `!RouteInfo`; once an HTLC is redeemed, the preimage travels back as `Fulfilled(ptr,x)`. One generic forwarding rule serves any number of hops:

```
rule Forward_HTLC:
    [ Route($Prev, $Me, ptrIn, y),
      !ChannelConnect(ptrOut, $Me, $Next),
      Mutex(ptrOut),
      In(%vIn), In(%fee), In(%vOut) ]
    --[ HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y),
        HTLCAdded(ptrOut, y),
        FeeCharged($Me, y, %fee),
        Eq(%vIn,  %vOut %+ %fee),
        Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me, $Next) ]->
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y,
      ↪ %vIn, %vOut, %fee),
      Route($Me, $Next, ptrOut, y),
      !RouteInfo($Me, $Next, ptrOut, y) ]
```

It consumes the incoming `Route` fact, produces the outgoing one, takes the slot of the outgoing channel, and stores in `IntermediaryPending` everything the hop needs to settle or be refunded later. `$Next` is any party with which `$Me` shares a channel, so routes are built hop by hop and have no fixed length. `HTLCForwarded` identifies the hop in our lemmas, `HTLCAdded` feeds restriction `OneHTLCPerHashPerChannel`, and `FeeCharged` records the fee the hop expects; the `Neq` facts make the forwarder distinct from its neighbours and the two channels distinct.

The settle round runs backwards (Appendix B-D). `Receiver_Create_Invoice` signs and sends a tagged invoice naming the receiver, the sender and the payment hash $y = h(x)$ of a fresh preimage x, kept private in `ReceiverInv`; `Sender_Offer_HTLC` checks the signature before offering the first HTLC. `Receiver_Fulfill_HTLC` consumes the last `Route` fact, checks that x hashes to y, emits `Redeemed(ptrIn,y)` and passes `Fulfilled(ptrIn,x)` upstream; `Intermediary_Settle` turns `IntermediaryPending` and `Fulfilled(ptrOut,x)` into `Fulfilled(ptrIn,x)`, recording `FeeEarned`. Both free `Mutex(ptrIn)`; `Sender_Settle` closes the payment (`SenderSettled`). Once `Open_Timeout` times a channel out (an untimed event, Sec. IV-A), `Sender_Timeout_Refund` and `Intermediary_Timeout_Refund` refund its pending HTLC without punishment (`Refunded`) and free the slot.

3) The Wormhole Rule: Besides the Dolev-Yao network and key compromise (`Compromise_Ltk`), the adversary has one routing rule:

```
rule AttackerSkip:
    [ Fulfilled(ptrExit, x),
      !RouteInfo(ExitAtt, Next, ptrExit, y)[-,no_precomp],
      !RouteInfo(EntryAtt, Skipped, ptrSkip, y)[-,no_precomp],
      !Compromised(ExitAtt),
      !Compromised(EntryAtt) ]
    --[ FulfilledProduced(ptrSkip, x),
        Skipped(ptrSkip, x),
        Eq(h(x), y) ]->
    [ Fulfilled(ptrSkip, x) ]
```

If two compromised parties, `ExitAtt` and `EntryAtt`, have added HTLCs for the same payment hash y on channels `ptrExit` and `ptrSkip`, `AttackerSkip` delivers the preimage released on `ptrExit` on `ptrSkip`. `EntryAtt` can then settle upstream, while the hops between `ptrSkip` and `ptrExit`, starting with `Skipped`, never see their outgoing HTLC fulfilled and can at best be refunded. The rule emits `Skipped(ptrSkip,x)` but neither `Redeemed` nor `FeeEarned`; it yields the wormhole trace of Sec. V-E (`Wormhole_Fee_Theft_Reachable`).

4) Amounts and Fees: Amounts and fees are modelled per hop only. `Forward_HTLC` reads the incoming amount, the fee and the outgoing amount from the network as natural numbers and requires the incoming amount to equal the outgoing amount plus the fee, so this balance holds at every hop by construction. `Sender_Offer_HTLC`, the invoice and the `Route` facts carry no amount, so amounts are not chained along the path, and the model says nothing about end-to-end value, such as the amount the receiver obtains or the total fee the sender pays. We keep per-hop fees because the wormhole is a fee theft: `FeeCharged` is emitted when a forwarder locks its outgoing HTLC, `FeeEarned` only when it later claims its incoming one, and in the wormhole trace the bypassed honest hop emits only the first, although the sender settles and the receiver is paid. A hop whose key is never compromised earns a fee only once its outgoing HTLC is redeemed (Lemma 5). The fee is one per-hop quantity, not Lightning's `base_fee + fee_rate` formula.

Three non-goals of Sec. IV-C show in the rules. `Forward_HTLC` has no `Wallet` premise and HTLCs do not change the balances in `StateA` and `StateB`, so a forward never fails for lack of funds. Route information is kept in local state, not in layered encryption: the sender chooses only the first hop, each forwarder its next hop among its own channels, and the wormhole appears purely as fee theft, as also characterised in [6]. The mirrors abstract the output structure of Fig. 2, and the two-stage HTLC claim becomes a single transition; for the channel balance, the second stage's CSV delay and revocation path are enforced by `Publish_Current_A/B`, `Claim_Funds_A/B` and the punishment rules.

B. Verification Results

Table I lists every verification run of our artifact, all obtained with Tamarin 1.10 (which reports the Maude version used, 3.2, as unsupported) and reproduced on a laptop with a 6 GB heap limit (`MEM=6G`). The default mode (`make verify`) proves the safety lemmas about channels and payments (Secs. V-C to V-E, Appendix A) and that, if the sender settles, the forwarders at positions 1 to 6 of a route are bypassed only if some key was compromised (Sec. VI-A). Its exists-trace lemmas show that the model is not vacuous (channel updates, cooperative closes, refunds and honest multi-hop payments are reachable), that the assumptions of our safety lemmas matter (a payment without a matching invoice can settle once the receiver's key is compromised; a compromised intermediary can lose), and that the wormhole is reachable. `make general` drops T1 to T3 and proves the bypass theorem for routes of any

TABLE I
VERIFICATION RUNS OF `lightning.spthy` (TAMARIN 1.10), IN THE ORDER OF `make all`; `aux` RUNS `lightning_aux.spthy` (TABLE III). A, E: ALL-TRACES AND EXISTS-TRACE LEMMAS.

Target	File, flags and content	A	E
`verify`	All restrictions: 32 safety, 13 hop-bypass, 10 exists-trace (1 replayed)	45	10
`general`	`-D=GENERAL`, `NO_*`: T1 to T3 removed; Theorem 2 (7 steps), 7 helpers, 16 `verify` lemmas re-proved	24	0
`aux`	`-D=PTLC`: countermeasure on a fixed route	1	4
	`-D=HTLC`: baseline, wormhole reachable	0	3
	`-D=CLOCK`: block clock	6	3
	`-D=CLTV`: CLTV-delta arithmetic	3	0
	`-D=TIMEOUT`: early-timeout race reachable	0	1
`necessity`	T3, T2 or T1 removed per run (Table II): 5 liveness lemmas, 600 s each: 10/15 verified, 5 without result; T3 counterexample (28 steps)	5×3	1
Total	`verify`, `general`, `aux`: 100/100 verified	79	21

length, for payments with a single offer per payment hash (Sec. VI-B); since removing restrictions only adds traces, this all-traces result also holds in the default mode.

a) The Linear Single-Spend Discipline: We enforce the no-double-spend rule using linear facts. Our first `Publish_Revoked_A` lacked the linear premise `RevokedCommitmentUnspent`, so punishment was gated only by the persistent `!RevokedSecret` and could fire again and again for the same revoked state: if a party cheats, the honest party can take the channel funds many times. Lemmas like `No_Punishment_Without_Cheating` still pass, because a cheat is still required; what they miss is that punishment should happen only once. Now `Revoke_Old_Secret` also creates this token for both parties, and `Publish_Revoked_A` (symmetrically `Publish_Revoked_B`) must consume it, so a revoked commitment is punished only once. We found the same critical flaw in our initial routing layer: with the persistent `!ChannelConnect` alone, one channel could add unboundedly many HTLCs on the same output. The linear slot `Mutex(ptr)` fixes it: produced once at opening, consumed by adding an HTLC and given back when that HTLC is redeemed or refunded, it lets a channel carry at most one pending HTLC at a time and be reused, each time with a payment hash not used on it before (`OneHTLCPerHashPerChannel`). Adding a linear premise can remove exists-trace witnesses, so we re-ran `lightning.spthy` after both fixes: all 55 lemmas of `make verify` are verified, including the ten exists-trace lemmas.

C. Protocol Properties

We state the guarantees of `lightning.spthy` as lemmas, each named after the TAMARIN property that establishes it. We apply honesty assumptions explicitly, in each lemma, not globally; only the restrictions of Sec. IV-B hold in every trace. Unless stated otherwise, every lemma here is proved in the default mode (`make verify`), under all nine restrictions, including T1 to T3. We sort all lemmas into ten groups, one per stage of a payment that they constrain (Table IV in Appendix B-E; the circled numbers in Figs. 3 and 4): channel layer and keys (0), invoice and preimage (1), lock (2) and release (3) causality, atomicity (4), fees (5), refund and griefing (6), wormhole (7), intermediary safety and timing (8), hop bypass (9). We present the central lemmas below; Appendix A states every lemma verbatim.

The first guarantee is atomicity: on a route with two forwarders and no key compromise, once the receiver has redeemed its HTLC, the sender can no longer be refunded.

Lemma 1 (Payment Atomicity)**.** *For any two consecutive forwarders F_1 and F_2 on a payment with hash y (the incoming HTLC of F_2 is the outgoing HTLC of F_1): if the outgoing HTLC of F_2 on y is redeemed and no party's key was ever compromised, then the incoming HTLC of F_1 on y is never refunded, by anyone (*`Payment_Atomicity_Under_Liveness`*, Appendix A-D).*

The second guarantee protects the intermediaries, who lock their own funds for a payment that is not theirs.

Lemma 2 (Intermediary Safety)**.** *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's long-term key was compromised (*`Intermediary_Never_Loses_Under_Liveness`*, Appendix A-H).*

Lemma 2 does not say when the key was compromised: a compromise after the refund also satisfies it. The strong form closes this gap.

Lemma 3 (Intermediary Safety, Strong Form)**.** *Under the premises of Lemma 2, that forwarder's key was compromised before its incoming HTLC timed out (*`Intermediary_Never_Loses_Strong`*, Appendix A-H).*

This matches T2 and T3, which bind only forwarders not compromised before their incoming HTLC times out. As `Forward_HTLC` is one generic rule, Lemma 1 holds for any two consecutive forwarders and Lemmas 2 and 3 for every forwarder, anywhere on a route of any length (under T1 to T3). Nothing is forwarded out of thin air:

Lemma 4 (Routing Causality)**.** *Every forward on hash y is preceded by the HTLC it forwards: either the sender's offer on the forwarder's incoming channel, or the predecessor's forward that locked that channel's HTLC (*`Forward_Requires_Incoming`*, Appendix A-B).*

This one lemma covers every hop of a route of any length, where a model with a fixed path needs one lemma per position (and an honest variant of each). It needs no honesty premise, because a forward consumes the linear `Route` fact, which only `Sender_Offer_HTLC` and `Forward_HTLC` produce. On the release side, every redemption stems from the receiver's fulfilment or from a forwarder settling that hop (`Redeemed_Source`), and every preimage with which a forwarder or the sender settles comes from an earlier redemption on that channel or an earlier wormhole skip (`Fulfilled_Source`, Sec. V-E). Finally, a forwarder is paid

only if its service succeeded; as amounts are not chained along the route (Sec. V-A4), this is a per-hop guarantee, not an end-to-end value claim.

Lemma 5 (Fees Only on Success)**.** *A forwarder whose key is never compromised earns the fee of a hop only after it has forwarded on that hop and its outgoing HTLC has been redeemed (*`Fee_Only_On_Successful_Forward`*, Appendix A-E).*

a) Non-vacuity: Safety lemmas are useless if the model cannot run at all, so we checked `exists-trace` witnesses (Appendices A-D and A-H): an HTLC is routed over two forwarders to the receiver, who fulfils it (`Multihop_Routing_Reachable`); payments over one and two forwarders settle end to end, with no key compromise and no refund: every forwarder claims its incoming HTLC and the sender settles (`Sanity_Honest_Payment_Completes`, `Sanity_Atomic_Payment_Completes`; the second satisfies the premises of Lemma 1); three channels among four different parties open with no compromise and no dispute (`Distinct_Parties_Configuration`); a compromised forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded (`Sanity_Compromised_Intermediary_Can_Lose`), so the premises of Lemmas 2 and 3 are satisfiable.

D. Timing and Liveness

Concrete block heights appear only in the auxiliary models of `lightning_aux.spthy` (Sec. IV-C). With `-D=CLTV`, a positive CLTV delta (restriction `DeltaPositive`) leaves every forwarder a non-empty claim window: a preimage revealed before the outgoing deadline is revealed before the incoming one (`Claim_Window_Nonempty`), also over two staggered hops (`Staggered_Path_Safe`). The four-rule model `-D=TIMEOUT` has no ordering restriction at all. As a two-sided check, Tamarin finds the early-timeout race there (`Early_Timeout_Race`, exists-trace, 6 steps: a forwarder's incoming HTLC times out and is refunded, and its outgoing HTLC is redeemed after that timeout), whereas in `lightning.spthy Timeout_Race_Blocked` (Appendix A-H) excludes it for every forwarder that is not compromised before its incoming timeout, even with any single one of T1, T2 and T3 removed (Table II).

We removed each liveness restriction in turn and reran the five lemmas that may depend on them (Table II). Without T3, Tamarin finds a counterexample with no key compromise (`Necessity_IMC_Honest_Intermediary_Loses`, exists-trace, 28 steps, Appendix A-H): the receiver redeems the outgoing HTLC of an intermediary that never claims its incoming HTLC, and the sender's HTLC to it is refunded. This honest intermediary loses, which violates Lemmas 2 and 3 (whose direct proof attempts without T3 gave no result within 600 s). T2 only orders the two timeouts, and no ordering rule can force an action: the claim is exactly T3, which is therefore load-bearing for intermediary safety. The ordering restrictions are not needed in this way: T1 is needed by none of the five lemmas, and T2 not for intermediary safety. From the cells without a result we conclude nothing: we do not claim that T2 is needed by `ForwardTimeGap` or Lemma 1, or T3 by Lemma 1.

TABLE II
Necessity of the liveness restrictions (`make necessity`: `-D=NECESSITY`, one restriction removed per column, 600 s per check). verified (n): proved in n steps; no result: neither a proof nor a counterexample within 600 s, from which we draw no conclusion; –: not checked.

Lemma	T1 removed `-D=NO_RBT`	T2 removed `-D=NO_OTF`	T3 removed `-D=NO_IMC`
`ForwardTimeGap`	verified (6)	no result	verified (6)
`Timeout_Race_Blocked`	verified (4)	verified (4)	verified (4)
Lemma 2 (intermediary)	verified (4)	verified (4)	no result
Lemma 3 (strong form)	verified (4)	verified (4)	no result
Lemma 1 (atomicity)	verified (10)	no result	no result
`Necessity_IMC_Honest_Intermediary_Loses` (exists-trace)	–	–	verified (28)

a) Liveness as Restrictions: Two of our three liveness assumptions, T2 and T3, are Tamarin restrictions of the form *if A happens, then B happens* (T1 only orders a redemption before a timeout); in T3, for instance, B is the redemption of the forwarder's incoming HTLC. A restriction removes every trace that violates it, including traces in which B is impossible rather than merely late, so a lemma proved under T3 says nothing about runs in which an honest forwarder *cannot* claim in time, which is where real timing attacks live. Likewise, T2 removes every trace in which an honest forwarder's incoming HTLC times out while its outgoing HTLC has not timed out earlier.

Which results depend on it. Removing T3 breaks intermediary safety (above). The 24 lemmas of `make general` (the bypass theorem for routes of any length, Sec. VI-B, its helpers and 16 lemmas of `make verify`, among them `Skipped_Requires_Compromise` and `Refund_Requires_Timeout`) are proved with all three liveness restrictions removed. The wormhole trace `Wormhole_Fee_Theft_Reachable` is verified with every restriction in place, so the restrictions do not hide it. All other all-traces lemmas of `make verify`, including the bypass theorems for positions 1 to 6, are proved with all three restrictions in place; apart from the five lemmas of Table II, we have not checked which of them hold without them.

A claim window exists, in a separate model. The Clock model (`-D=CLOCK`, Table I, Appendix A-K) gives channels concrete deadlines on a block clock, has no liveness restriction and proves `Claim_Window_Exists`: if a forwarder's outgoing HTLC is redeemed and its incoming HTLC is refunded, the redemption happens at a strictly lower block height than the refund, so the forwarder had a non-empty window in which to claim. It also proves `Loss_Implies_Skipped_Claim`: a forwarder that loses in

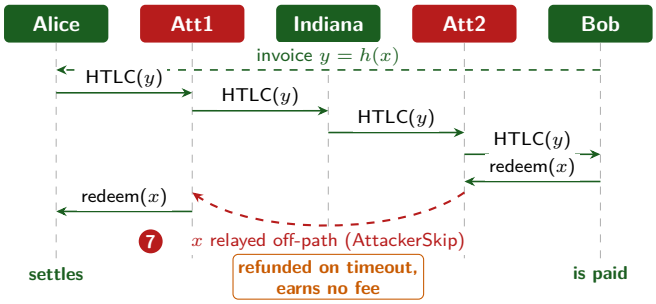


Fig. 4. Wormhole (`Wormhole_Fee_Theft_Reachable`); red: compromised, green: honest; drawn in causal order (the lemma fixes no order).

this way never claimed its incoming HTLC. The Clock model is not composed with `lightning.spthy`, so it supports T3 but does not discharge it.

Alternatives. A more faithful treatment would replace the restrictions by the progress semantics of Backes et al. [23]: roughly, honest processes that can continue without input from the adversary eventually do, so traces in which the forwarder cannot act are kept, not pruned. Tamarin 1.10 implements it for Sapic+ processes (option `translation-progress`), into which the routing layer would have to be rewritten. A cheaper first step is to make T1, T2 and T3 premises of only the lemmas that need them, so that they no longer prune traces for every other lemma. We leave both to future work (Sec. VIII).

E. Griefing and the Wormhole

`Refund_Possible` shows a griefing trace: the sender offers an HTLC against an invoice, nobody forwards it, and it is refunded, so the payment never settles; no key is compromised. Since a refund requires a timeout (`Refund_Requires_Timeout`), the capital stays locked until then. This witness refunds the sender, not an intermediary. We report it as reachable griefing (denial of service), not as a new attack (group 6, the circled 6 in Fig. 3b).

The wormhole attack [6] is an instance of the agent-skipping attack formalised symbolically by [22]; we do not claim it as new. First, after the linear single-spend fix (Sec. V-B), it is a soundness check: the model must still admit this behaviour, or the fix would be too strong. Second, the model shows who can mount the attack: two compromised parties on the same payment, which relay x around the hops between them (`AttackerSkip`, Sec. V-A3). `Wormhole_Fee_Theft_Reachable` (exists-trace, verified by replaying a saved 116-step proof; Appendix A-G) is the five-party trace of Fig. 4 (circled 7). Bob invoices Alice for $y = h(x)$; the HTLC is locked hop by hop from Alice through Att1, Indiana and Att2 to Bob. Bob redeems the HTLC of Att2 with x and is paid; `AttackerSkip` relays x from Att2 to Att1, off the path, and Att1 claims the HTLC of Alice, so Alice settles, while Indiana's outgoing HTLC to Att2 is refunded on timeout and Indiana earns no fee. The lemma requires that Alice, Bob and Indiana are never compromised; in the trace, Att1 and Att2 are, as `AttackerSkip` requires. An honest hop is thus bypassed although the payment settles. Conversely, a skip requires a key compromise:

Lemma 6 (Skips Require Compromise)**.** *Every wormhole skip happens after some party's long-term key was compromised (*`Skipped_Requires_Compromise`*, Appendix B-F).*

Lemma 6 does not exclude the wormhole; it says that the wormhole needs a compromise.

VI. Bypass Resistance for Routes of Any Length

The wormhole of Sec. V-E lets a payment settle while an honest forwarder between two compromised parties is bypassed: its outgoing HTLC is refunded and it earns no fee. We now show that in our model a bypass requires key compromise: for the forwarder at each of the positions 1 to 6 of a route, and, for payments whose hash is offered on a single channel, for a forwarder at any position of a route of any length. Secs. VI-A to VI-D concern the main theory `lightning.spthy`; there is no separate N-hop theory. HTLC offers between channel peers are not network messages but the private, single-use fact `Route`, which each application of the one generic rule `Forward_HTLC` consumes and re-creates for the next hop (Sec. V-A2): with the offer as a public, signed message, Tamarin treats every party as a potential sender when tracing a forwarded message, and the analysis of routes of unbounded length did not terminate (what this takes away from the adversary: Sec. VIII).

A. Bypass Requires Compromise

The only rule that produces a `Fulfilled` fact for an HTLC that was not redeemed is `AttackerSkip` (Sec. V-A3), which requires the parties at both ends of the skip to be compromised; `Fulfilled_Source` states that every `Fulfilled` fact used in a settlement stems from an earlier redemption or skip, and every skip follows a compromise (Lemma 6). A forwarder is *bypassed* when the payment settles at the sender while the forwarder's own hop is refunded; `Intermediary_Timeout_Refund` records such a refund as `RefundHop(ptrIn,ptrOut,y)`. For the forwarders at the first six positions of the sender's route we prove, in the full model with T1, T2 and T3 in place:

Theorem 1 (Bypass Resistance at Positions 1 to 6)**.** *Let $1 \le k \le 6$. If a sender settles a payment with preimage x on its own channel, the HTLC on $h(x)$ is forwarded hop by hop from that channel by forwarders $F_1, \ldots, F_k$, and the hop of F_k is refunded, then some party's long-term key was compromised.*

Its six instances, `First_Hop_Skip_Requires_Compromise` to `Hop6_Skip_Requires_Compromise` (Appendix A-I), spell out the route: `SenderSettled` on the sender's channel `pSF`, a chain of k `HTLCForwarded` facts on $h(x)$ in which F_1 forwards the HTLC on `pSF` and each F_{i+1} the outgoing

HTLC of F_i, and `RefundHop` for the hop of F_k. With their seven helper lemmas (group 9 of Table IV), they are the 13 hop-bypass lemmas of `make verify`.

The wormhole trace `Wormhole_Fee_Theft_Reachable` (Fig. 4), with Indiana as the second forwarder (F_1 is Att1, F_2 is Indiana), satisfies the premise of `Second_Hop_Skip_Requires_Compromise`; the compromise in the conclusion is that of Att1 and Att2, which `AttackerSkip` requires, so the conclusion cannot be dropped.

B. Routes of Any Length

Each fixed-position theorem names the route up to the bypassed forwarder and therefore covers one position only; the general theorem quantifies over any forward of the payment hash instead.

Theorem 2 (Bypass Resistance at Any Position)**.** *If a sender settles a payment with preimage x on its channel p_{SF}, every HTLC on $h(x)$ is offered on p_{SF}, and some forwarder of an HTLC on $h(x)$, at any position, has its hop refunded, then some party's long-term key was compromised.*

```
lemma Any_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
       (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f))
      ↪ ∧
      (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p =
      ↪ pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

The last conjunct of the premise, every offer for $h(x)$ is on `pSF`, is the *single-offer premise*. No position appears in the statement: `Me` is any forwarder of $h(x)$, at any distance from the sender.

C. Proof

The proof rests on seven helper lemmas, proved in the same run. `Forward_Parent`: the incoming HTLC of every forward was offered by a sender or forwarded by the previous hop. `Root_First`: every forward of a payment hash is, or is preceded by, a forward whose incoming HTLC was offered by a sender. `Next_Is_Child`: under the single-offer premise, if `Prev` forwarded an HTLC to `Me`, every later forward of the hash is `Me`'s forward of that HTLC or comes after it. `Hop_Redeem_Propagates` and `Sender_Channel_Redeemed`: without compromise, if a forwarder's incoming HTLC is redeemed, so is its outgoing one, and if the sender settles, its own channel was redeemed. `Route_Redeemed_Gen`: without compromise and under the single-offer premise, redemption of the incoming HTLC of a forward carries over to the incoming HTLC of every forward of the hash that does not come earlier. `Route_Redeemed_No_Compromise` combines it with `Sender_Channel_Redeemed` and `Root_First`: without compromise and under the single-offer premise, if the sender settles, the incoming HTLC of every forward of the hash is redeemed. `Root_First`, `Next_Is_Child` and `Route_Redeemed_Gen` are proved by induction over traces; their proof scripts, and those of `Route_Redeemed_No_Compromise` and of the theorem, are stored in `lightning.spthy`, and the other helpers are proved automatically.

The theorem then takes 7 proof steps. Assume no compromise. By `Route_Redeemed_No_Compromise`, the bypassed forwarder's incoming HTLC `pIn` is redeemed. By `Redeemed_Source`, this redemption is either the settlement of a forward from `pIn`, which `Forward_Unique_Per_Incoming`, `SettledHop_From_Forward` and `No_Settle_And_Refund_Same_Hop` exclude given `RefundHop(pIn, pOut,h(x))`, or a fulfilment by the receiver, which `No_Fulfill_On_Forwarded_Incoming` excludes.

a) General Mode and Trace Inclusion: The theorem is proved in a separate mode of the same theory, `make general` (flags in Appendix A), which enables the general lemmas and removes T1, T2 and T3. Removing a restriction only adds traces, so an all-traces property proved in this mode also holds in the full model. The run proves the theorem together with all 23 `[reuse]` lemmas available in this mode (24/24 verified), so no lemma proved under the removed restrictions is reused (`ForwardTimeGap`, derived from T2, is not a `[reuse]` lemma in this mode). Bypass resistance for single-offer payments therefore does not depend on T1, T2 or T3.

D. Caveats

Single-offer premise and liveness restrictions. Theorem 2 is proved only for payments whose hash is offered on a single channel; if the same hash is offered on several channels, several routes carry it, and the theorem says nothing about a forwarder on such a payment. The fixed-position theorems do not need the premise, because they spell out the route from the settling sender's channel; for such payments they cover positions 1 to 6 of that route. They are proved with T1, T2 and T3 in place, and we have not checked them without, so for such payments bypass resistance at positions 1 to 6 rests on the liveness restrictions; only Theorem 2 (single-offer payments) is shown to hold without them.

Weak conclusion. All bypass theorems conclude only that some party's long-term key was compromised at some point of the trace: not which party (e.g., a neighbour of the bypassed forwarder, as in the wormhole), and not that the compromise precedes the settlement or the refund. Only `Skipped_Requires_Compromise` orders the compromise before the skip.

E. Countermeasure: PTLC

The wormhole works because every channel of an HTLC route is locked with the same hash $h(x)$: once x is known at one point of the route, a compromised party can use it on a channel further upstream. Malavolta et al. [6] replace the shared lock by anonymous multi-hop locks, the basis of point time-locked contracts (PTLCs): every channel has its own lock, and a forwarder can compute the secret of its incoming lock only from the secret of its outgoing

lock and its own tweak, chosen by the sender. We compare both designs in two small models of `lightning_aux.spthy`, `-D=PTLC` and the baseline `-D=HTLC`, that differ only in how the channel secrets are derived (`Sender_Setup`) and in how an honest hop obtains its incoming secret (`Hop_Settle`).

a) Model: Both models fix the route $\mathrm{S} \to \mathrm{H1} \to \mathrm{H2} \to \mathrm{H3} \to \mathrm{R}$ over channels `c0` to `c3`. `Sender_Setup` draws a fresh secret x and per-hop tweaks t_1, t_2, t_3 and derives the channel secrets by a hash chain $s_{in} = h(\langle s_{out}, t \rangle)$: $s_3 = x$ and $s_{i-1} = h(\langle s_i, t_i \rangle)$ for $i = 3, 2, 1$; channel c_i is locked with $h(s_i)$; an honest hop derives its incoming secret from its outgoing one with its tweak (`Hop_Settle`; both rules in Appendix B-G). The hash chain is a symbolic stand-in for the additive tweak of PTLCs built from adaptor signatures [6], $s_{in} = s_{out} + t$: in both, deriving a hop's incoming secret requires that hop's tweak. In the HTLC baseline, $s_0 = s_1 = s_2 = s_3 = x$, i.e., one hash lock $h(x)$ on every channel, and `Hop_Settle` passes the outgoing secret on unchanged. The adversary is the same in both models: Dolev-Yao plus compromise of any party at any time (rule `Compromise`). A compromised hop leaks its tweak (`Leak_Tweak`); every secret used to claim a channel is published, as it would be on chain; a compromised payee may claim its channel with any value the adversary knows and that opens the lock (`Adversary_Claim`). There is no skip rule: a wormhole has to be built from adversary knowledge alone. Timing is abstracted: an open HTLC may be refunded at any time (`Refund`).

Lemma 7 (No Wormhole with Per-Hop Locks)**.** *In the PTLC model, as long as S and H2 are uncompromised, the sender's channel `c0` is never redeemed while H2's outgoing channel `c2` is refunded, whichever of H1, H3 and R are compromised (`No_Wormhole_Around_Honest_Hop`, all-traces, verified, 25 steps).*

With the shared hash lock, the same situation is reachable (`Wormhole_Around_Honest_Hop`, exists-trace, verified, 12 steps); both statements are printed in Appendix A-K. Two control lemmas show that the PTLC result is not vacuous. `Control_Refund_Of_H2_Outgoing` (exists-trace, 6 steps): H2's outgoing HTLC can be refunded while H2 is honest, so the refund in the premise is not excluded by the model itself. `Control_Wormhole_If_H2_Compromised` (exists-trace, 18 steps): once H2 itself is compromised, the bypass is reachable with S still uncompromised, so the safety proof depends on H2 being honest. Sanity lemmas in both models show that a payment completes end to end without any compromise (`Honest_Payment_Completes`: 10 steps in PTLC, 10 in HTLC) and with both neighbours H1 and H3 of H2 compromised (`Colluders_Present_Payment_Completes`: 14 steps in PTLC, 8 in HTLC). All eight lemmas are verified by `make aux` (PTLC 5/5, HTLC 3/3).

b) Limitations: The two models are separate from `lightning.spthy`: a fixed route with three intermediaries and one honest hop under attack, no channel layer, abstract timing, and an honest `Sender_Setup` that chooses all locks and tweaks consistently; the safety lemma also assumes that the sender is never compromised. The hash chain abstracts the group operation of the adaptor-signature construction and does not model the algebra of [6] (an XOR encoding exhausted memory in Tamarin). We therefore claim the contrast between the two lock designs on this route, not a PTLC result for routes of any length.

VII. Related Work

Grundmann and Hartenstein [8] model-check a stepwise refinement of the Lightning Network in TLA+; model checking explores a bounded state space. Our Tamarin proofs are symbolic and unbounded: subject to the restrictions of our model (Sec. IV-B), including three liveness restrictions, they cover any number of parties, channels, channel updates and payments over routes of any length. [9] presents a formally verified Lightning Network.

Kiayias and Litos [24] give a composable security treatment of the Lightning Network: they define an ideal functionality for a payment network and prove, in the universal composability framework, that the Lightning protocol realises it. Their guarantee is computational and composable, and their proof is by hand. Ours are symbolic but machine-checked by Tamarin; they include bypass resistance for routes of any length (for single-offer payments), and they contrast the wormhole under HTLCs with its absence under PTLC-style locks on a fixed route.

Malavolta et al. [6] describe the wormhole attack on HTLC-based multi-hop payments and propose anonymous multi-hop locks as a countermeasure; their instantiation with adaptor signatures is the basis of PTLCs. Adaptor signatures also underlie the generalised channels of Aumayr et al. [7], built from limited blockchain scripts. Our main model reproduces the wormhole (Sec. V-E), and our auxiliary models check the countermeasure symbolically on a fixed route (Sec. VI-E). Smith et al. [22] formalise agent-skipping attacks on message-forwarding protocols; the wormhole is an instance of such an attack. Our bypass theorems (Sec. VI) state the corresponding security property for Lightning routing: a forwarder at positions 1 to 6 of a route, or at any position for single-offer payments, can be skipped only if some long-term key is compromised.

Backes et al. [23] reason about liveness in cryptographic protocols with a progress semantics, in which honest processes that can continue without input from the adversary eventually do, and apply it to fair exchange. We impose liveness as trace restrictions instead (Sec. V-D).

VIII. Limitations

- *Abstract timing, liveness as restrictions* (Sec. V-D). The main model has no clock: `Open_Timeout` lets a channel time out at any point allowed by the restrictions, and T1, T2 and T3 (the last two only for forwarders not compromised before their incoming HTLC times out) stand in for CLTV deadlines and timely claiming.

They prune every trace that violates them, including traces in which an uncompromised forwarder could not have claimed in time; results proved under them say nothing about such runs. Only the 24 lemmas of `make general` are proved with all three removed; the necessity runs remove one at a time, and T3 is necessary for intermediary safety (Sec. V-D). Block heights appear only in the auxiliary CLOCK and CLTV models, which are not composed with `lightning.spthy`. The progress semantics of [23] would keep the pruned traces, but Tamarin 1.10 offers it only for Sapic+ processes.

- *Scope of the bypass and PTLC results* (Secs. VI-D and VI-E). Theorem 2 needs the single-offer premise; for a hash offered more than once, only positions 1 to 6 are covered, under T1, T2 and T3. No bypass theorem says whose key is compromised, or when. The countermeasure is checked only on a fixed route with three intermediaries, with honest lock set-up, an honest sender and a hash chain in place of the adaptor-signature algebra.
- *No end-to-end amounts.* Each `Forward_HTLC` requires its incoming amount to be the outgoing amount plus the fee, but the offer, the invoice and the `Route` fact carry no amount. Amounts are therefore not chained along the path, and we make no end-to-end claims about value, e.g., that the receiver obtains the invoiced amount or that the sender pays exactly the path fees.
- *HTLCs are not part of the commitment transactions.* The channel and routing layers share the theory, the adversary and the channel topology (HTLCs are added only on opened channels, `HTLC_On_Opened_Channel`), but settling or refunding an HTLC does not update the channel state. That each HTLC is resolved at most once is assumed as the restriction `OneResolutionPerHTLC`, not derived from the channel layer; we do not claim a composition theorem. A channel carries at most one pending HTLC at a time (`Mutex(ptr)`).
- *Routing abstraction, no privacy.* Offers travel as the private linear fact `Route`, so the network adversary cannot inject, replay or modify offers between peers; it chooses amounts and fees, controls the delivery of invoices and can compromise keys. It never learns a preimage (`Preimage_Never_Known`): in the main model the wormhole comes from the explicit rule `AttackerSkip`, which needs both ends of the skip compromised, rather than from adversary knowledge. There is no onion routing: every forwarder knows its predecessor, its successor and the payment hash, and the next hop is any party with which it shares a channel. We make no privacy claims.

IX. Conclusion

We presented one Tamarin theory of the Lightning Network that covers the channel layer and multi-hop HTLC routing over routes of any length, against a Dolev-Yao adversary that can also compromise long-term keys. We proved that only cheating or compromised parties are punished, that settlements are authenticated by the receiver's invoice and that preimages stay secret; under three liveness restrictions, an uncompromised intermediary never has its outgoing HTLC redeemed while its incoming one is refunded, and payments are atomic in runs without key compromise. The assumption that intermediaries claim in time is necessary: without it, Tamarin finds a run without any compromise in which an honest intermediary loses. The wormhole is a Tamarin-checked trace in which two compromised parties bypass an honest one, which is refunded and earns no fee. Conversely, a bypass requires some key compromise: under the liveness restrictions at positions 1 to 6 of any route, and, for payments whose hash is offered on a single channel, at every position even without them. On a fixed route, per-hop locks in the style of PTLCs rule out the bypass of an honest hop when the sender is honest, whereas a shared hash lock admits it. Next, we want to replace the liveness restrictions by the progress semantics of [23], so that runs in which an honest party cannot act are analysed rather than pruned, and to derive `OneResolutionPerHTLC` from the channel layer.

Artifact Availability

The artifact, `lightning.spthy`, `lightning_aux.spthy` (the five auxiliary models) and a `Makefile`, is available as supplementary material. `make all` runs `make verify` (55/55 lemmas verified), `make general` (24/24), `make aux` (21/21) and `make necessity`, in this order (Table I). All results were obtained with Tamarin 1.10 and reproduced on a laptop with a 6 GB heap (`MEM=6G`); `make verify-one` proves one lemma per run for machines with less memory. Tamarin 1.10 reports the Maude version used, 3.2, as unsupported; all results were obtained with it. Every run also reports that Tamarin's derivation checks, part of its well-formedness checks, timed out; this warning does not concern the proofs.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," p. 21260, 2008.

[2] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," 2016.

[3] (2026) Lightning Network Statistics | 1ML – Lightning Network Search and Analysis Engine – Bitcoin mainnet. [Online]. Available: https://web.archive.org/web/20260810134919/https://1ml.com/statistics

[4] "Lightning/bolts," Lightning Network, 2026. [Online]. Available: https://github.com/lightning/bolts

[5] S. Tikhomirov, P. Moreno-Sanchez, and M. Maffei, "A Quantitative Analysis of Security, Anonymity and Scalability for the Lightning Network," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2020, pp. 387–396.

[6] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability," in *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, 2019.

[7] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures," in *Advances in Cryptology – ASIACRYPT 2021*, M. Tibouchi and H. Wang, Eds. Springer International Publishing, 2021, vol. 13091, pp. 635–664.

[8] M. Grundmann and H. Hartenstein, "Security of the lightning network: Model checking a stepwise refinement with TLA+," in *Integrated Formal Methods*, F. Damiani and M. Farrell, Eds. Springer Nature Switzerland, 2026, pp. 313–335.
[9] G. , R. Stefański, and O. S. T. Litos, "A Formally Verified Lightning Network," in *Financial Cryptography and Data Security*, C. Garman and P. Moreno-Sanchez, Eds. Springer Nature Switzerland, 2026, pp. 3–20.
[10] D. Dolev and A. Yao, "On the security of public key protocols," vol. 29, no. 2, pp. 198–208, 1983.
[11] B. Blanchet, "A computationally sound mechanized prover for security protocols," in *2006 IEEE Symposium on Security and Privacy (S&P'06)*, 2006, pp. 15 pp.–154.
[12] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, "Computer-Aided Security Proofs for the Working Cryptographer," in *Advances in Cryptology – CRYPTO 2011*, ser. Lecture Notes in Computer Science, P. Rogaway, Ed. Springer, 2011, pp. 71–90.
[13] D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and S. Moreau, "An Interactive Prover for Protocol Verification in the Computational Model," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 537–554.
[14] S. Jeanteur, L. Kovács, M. Maffei, and M. Rawson, "CryptoVampire: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model." IEEE Computer Society, 2024, pp. 3165–3183.
[15] B. Blanchet, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," in *14th IEEE Computer Security Foundations Workshop (CSFW-14)*. IEEE Computer Society, 2001, pp. 82–96.
[16] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, N. Sharygina and H. Veith, Eds. Springer, 2013, pp. 696–701.
[17] B. Lipp, B. Blanchet, and K. Bhargavan, "A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 231–246.
[18] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A Comprehensive Symbolic Analysis of TLS 1.3," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. Association for Computing Machinery, 2017, pp. 1773–1788.
[19] G. Andresen, "BIP 11: M-of-N Standard Transactions," 2011-10, 2011. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki
[20] E. Lombrozo, J. Lau, and P. Wuille, "BIP 141: Segregated witness (consensus layer)," 2015-12, 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
[21] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn, and A. Miller, "Pisa: Arbitration Outsourcing for State Channels," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, ser. AFT '19. Association for Computing Machinery, 2019, pp. 16–30.
[22] Z. Smith, H. Jonker, S. Mauw, and H. Lee. (2022) Modelling Agent-Skipping Attacks in Message Forwarding Protocols.
[23] M. Backes, J. Dreier, S. Kremer, and R. Künnemann, "A Novel Approach for Reasoning about Liveness in Cryptographic Protocols and its Application to Fair Exchange," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017.
[24] A. Kiayias and O. S. T. Litos, "A Composable Security Treatment of the Lightning Network," in *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, 2020.

Appendix A
Lemma Catalogue

This appendix renders every lemma of Table IV in natural language and gives its exact Tamarin statement, so that the table can be checked against the theories. Two kinds of statements are given elsewhere: that of `Skipped_Requires_Compromise` is in Appendix B-F, and the seven helper lemmas of the general theorem are listed by name. The last subsection covers the lemmas of the auxiliary models.

Unless stated otherwise, every lemma below is verified in `lightning.spthy`, the main model (channel layer, multi-hop routing in which one generic rule `Forward_HTLC` serves every hop of a route of any length, and the wormhole rule `AttackerSkip`). Lemmas of the auxiliary models are verified in `lightning_aux.spthy`, one model per preprocessor flag (`-D=PTLC`, `-D=HTLC`, `-D=CLOCK`, `-D=CLTV`, `-D=TIMEOUT`). Every listing is copied verbatim from these files, with the proof script omitted. The subsections follow the groups of Table IV: groups 1 to 9 first, then group 0 (channel layer and keys). We use the same naming convention as the table: A for an all-traces safety property, E for an exists-trace witness (reachability or non-vacuity).

All results were obtained with Tamarin 1.10 and are reproduced by `make all`. `make verify` proves all 55 lemmas of the full model, i.e., with all nine restrictions in place, including the liveness restrictions T1, T2 and T3 (32 all-traces safety lemmas, 13 hop-bypass lemmas, 9 exists-trace lemmas found automatically, and `Wormhole_Fee_Theft_Reachable`, replayed from a saved 116-step proof). `make general` proves the 24 lemmas of the any-length theorem in the model without the three liveness restrictions. `make aux` proves the 21 lemmas of `lightning_aux.spthy`, and `make necessity` removes one liveness restriction at a time (Table II).

Running a single check. A lemma is checked on its own by calling Tamarin on its file with `--prove=` followed by the lemma name, and with the flags of its mode; an auxiliary model is proved as a whole with its flag and `--prove`. For example, for a lemma of `make verify`, a lemma of the general mode and the CLTV model:

```
make all
tamarin-prover lightning.spthy --prove=ForwardTimeGap
tamarin-prover lightning.spthy -D=GENERAL -D=NO_IMC \
  -D=NO_OTF -D=NO_RBT -D=NECESSITY \
  --prove=Any_Hop_Skip_Requires_Compromise
tamarin-prover lightning_aux.spthy -D=CLTV --prove
```

A single check uses the preceding lemmas marked `[reuse]` as assumptions; `make verify` and `make general` prove those lemmas too. `Wormhole_Fee_Theft_Reachable` is checked with `--lemma=` in place of `--prove=`, which replays the proof saved in the file. The TIMEOUT model also needs `--heuristic=c` and `--stop-on-trace=seqdfs`. If memory is short, `make verify-one` checks the lemmas of `make verify` one per run, and `MEM` sets the heap limit of Tamarin (6 GB by default).

Action facts. Throughout, action facts carry their intended meaning from Sec. V-A; $y = h(x)$ is a payment hash and `ptr` a channel.

- `InvoiceCreated(R, S, y)`: receiver `R` issued an invoice for y to sender `S`. The invoice, the tuple of the tag `'invoice'`, `R`, `S` and y, is signed by `R` and carries no amount.
- `SecretPreimage(R, x)` and `PreimageReleased(R, x)`: `R` generated the preimage x, and `R` released it.

- `HTLCOffered(S, F1, ptr, y)`: the sender `S` offered an HTLC on y to its first forwarder `F1` on channel `ptr`. `OfferFor(S, R, ptr, y)` annotates the same step with the receiver `R` of the invoice.
- `HTLCAdded(ptr, y)`: an HTLC on y was added to channel `ptr`, by an offer or by a forward.
- `HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y)`: forwarder `Me`, holding an incoming HTLC from `Prev` on `ptrIn`, added an outgoing HTLC to `Next` on `ptrOut` (one rule for every hop). `FeeCharged(Me, y, %fee)` is the fee `Me` charges at that forward.
- `HTLCFulfilled(R, Prev, ptr, y)`: the receiver redeemed its incoming HTLC.
- `IntermediaryClaimed(Me, Prev, ptrIn, x)` and `FeeEarned(Me, ptrIn, ptrOut, y, %fee)`: forwarder `Me` redeemed its incoming HTLC with x, and earned its fee.
- `SenderSettled(S, F1, R, ptr, x)`: the sender settled its HTLC with x.
- `Redeemed(ptr, y)` and `Refunded(Up, ptr, y)`: the HTLC on `ptr` was redeemed; or it was refunded, after its timeout, to `Up`, the party that had offered it.
- `TimedOut(ptr)`: the HTLC timeout of channel `ptr` was reached.
- `SettledHop(ptrIn, ptrOut, y)` and `RefundHop(ptrIn, ptrOut, y)`: annotations; the hop from `ptrIn` to `ptrOut` settled, or had its outgoing HTLC refunded.
- `UsedFulfilled(ptr, x)`: annotation; a settlement consumed the fulfilment of the HTLC on `ptr`.
- `FulfilledProduced(ptr, x)`: a fulfilment for `ptr` was produced by an intermediary settlement or by a skip.
- `Skipped(ptr, x)`: annotation of `AttackerSkip`; compromised parties relayed x onto `ptr` (wormhole).
- `ChannelOpen(A, B, ptr)`: `A` and `B` opened channel `ptr`.
- `KeyGen(A, ltk)` and `LtkCompromised(A)`: `A` generated its long-term key `ltk`; the long-term key of `A` was compromised.
- $K(x)$: the adversary knows the term x.

The action facts of the channel layer are explained in Appendix A-J.

A. Invoice and Preimage (Group 1)

a) `Invoice_Authenticates_Settlement` (A): The sender can settle an HTLC only if the payment hash corresponds to an invoice the receiver actually issued, at a strictly earlier time; the only way around this is for the receiver's long-term key to have been compromised (strictly earlier as well). In short, a settlement is legitimate only when a genuine invoice backs it; a settlement without such an invoice is attributable to a compromised receiver.

```
lemma Invoice_Authenticates_Settlement:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> (Ex #j. InvoiceCreated(R, S, h(x)) @ #j & #j < #i)
       | (Ex #r. LtkCompromised(R) @ #r & #r < #i)"
```

b) `Forged_Invoice_Requires_Key_Compromise` (E): Reachability witness. There is a trace in which the sender settles an HTLC for receiver `R` although `R` never created an invoice for $h(x)$; the long-term key of `R` is compromised in this trace, and no forwarding takes place. Together with `Invoice_Authenticates_Settlement`, which requires such a compromise to precede the settlement, this shows that the compromise case of that lemma is not vacuous.

```
lemma Forged_Invoice_Requires_Key_Compromise:
    exists-trace
    "Ex S F1 R ptr x #i #r.
        SenderSettled(S, F1, R, ptr, x) @ #i
      & LtkCompromised(R) @ #r
      & not (Ex #j. InvoiceCreated(R, S, h(x)) @ #j)
      & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev,
       ↪ Next, pi, po, y) @ #u)"
```

c) `Invoice_Has_Secret_Preimage` (A): Every invoice is created together with the receiver's secret preimage of its hash.

```
lemma Invoice_Has_Secret_Preimage [reuse]:
    "All R S x #i.
        InvoiceCreated(R, S, h(x)) @ #i
     ==> SecretPreimage(R, x) @ #i"
```

d) `Preimage_Secret_Until_Released` (A): As long as the receiver holds a secret preimage and has not released it, the adversary does not know it: the first moment the adversary can learn the preimage is strictly after the receiver has released it. Formally, if the receiver generated a secret preimage and the adversary later learns it, then the receiver must have released that preimage at a strictly earlier point in the trace. In this model the preimage is never sent to the network: fulfilments travel between hops as state facts, and `AttackerSkip` relays them without the adversary learning x. Hence `Preimage_Never_Known` holds, and this lemma holds trivially.

```
lemma Preimage_Secret_Until_Released:
    "All R x #i #j.
        SecretPreimage(R, x) @ #i
      & K(x) @ #j
      & #i < #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #j"
```

e) `Preimage_Never_Known` (A): The adversary never learns a receiver's preimage (x is never output in this model); `Preimage_Secret_Until_Released` therefore holds trivially.

```
lemma Preimage_Never_Known:
    "All R x #i. SecretPreimage(R, x) @ #i ==> not (Ex #j. K(x)
     ↪ @ #j)"
```

f) `Invoice_Released_Once` (A): The receiver releases a given preimage at most once: if the same receiver releases the same preimage at two times in a trace, those two times coincide, i.e., there is a single release event. This holds because `Receiver_Fulfill_HTLC` consumes the linear fact `ReceiverInv(R, S, x)`, which `Receiver_Create_Invoice` creates once per fresh preimage x.

```
lemma Invoice_Released_Once [reuse]:
    "All R x #i #j.
        PreimageReleased(R, x) @ #i
      & PreimageReleased(R, x) @ #j
```

```
    ==> #i = #j"
```

B. Lock Causality (Group 2)

a) `Forward_Requires_Incoming` (A): A forwarder adds an outgoing HTLC only if an HTLC on the same hash was added earlier to its incoming channel, either offered to it by a sender or forwarded to it by its predecessor. The lemma covers every hop of a route of any length and needs no honesty assumption. It is Lemma 4.

```
lemma Forward_Requires_Incoming [use_induction]:
    "All Me Prev Next ptrIn ptrOut y #i.
        HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #i
     ==> (Ex S #j. HTLCOffered(S, Me, ptrIn, y) @ #j & #j < #i)
       | (Ex P0 pin0 #j. HTLCForwarded(Prev, P0, Me, pin0,
     ↪ ptrIn, y) @ #j & #j < #i)"
```

b) `HTLC_On_Opened_Channel` (A): An HTLC is only ever added on a channel that has been opened beforehand: whenever an HTLC is added on a channel, two parties opened that channel at a strictly earlier time. `HTLCAdded` is emitted by `Sender_Offer_HTLC` and `Forward_HTLC`.

```
lemma HTLC_On_Opened_Channel [reuse, use_induction]:
    "All ptr y #i.
        HTLCAdded(ptr, y) @ #i
     ==> Ex X Y #j. ChannelOpen(X, Y, ptr) @ #j & #j < #i"
```

c) `Offer_Requires_Invoice` (A): An HTLC is offered for receiver `R` only against an invoice that `R` created earlier, or the long-term key of `R` is compromised at some point of the trace.

```
lemma Offer_Requires_Invoice:
    "All S R ptr y #o.
        OfferFor(S, R, ptr, y) @ #o
     ==> (Ex #j. InvoiceCreated(R, S, y) @ #j & #j < #o)
       | (Ex #r. LtkCompromised(R) @ #r)"
```

C. Release Causality (Group 3)

a) `Claim_Requires_Release` (A): An intermediary can claim its incoming HTLC with a preimage only after (strictly later than) the receiver has released that preimage: the release of the preimage is a prerequisite for any intermediary settlement.

```
lemma Claim_Requires_Release [use_induction]:
    "All MidP InP ptrIn R x #i #j.
        IntermediaryClaimed(MidP, InP, ptrIn, x) @ #i
     & SecretPreimage(R, x) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

b) `Settle_Requires_Receiver_Release` (A): When the sender settles an HTLC on a genuine invoice, the receiver must have released the corresponding preimage strictly earlier: a settlement never precedes the receiver's release of the preimage.

```
lemma Settle_Requires_Receiver_Release:
    "All S F1 R ptr x #i #j.
        SenderSettled(S, F1, R, ptr, x) @ #i
     & InvoiceCreated(R, S, h(x)) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

c) `Redeem_Requires_Receiver_Release` (A): Every redeemed HTLC on y comes with a receiver's release of a preimage x with $h(x) = y$.

```
lemma Redeem_Requires_Receiver_Release [use_induction]:
    "All ptr y #i.
        Redeemed(ptr, y) @ #i
     ==> Ex R x #k. PreimageReleased(R, x) @ #k & h(x) = y"
```

d) `Fulfilled_Requires_Release` (A): Every fulfilment produced by an intermediary settlement or a skip is preceded by a receiver's release of x.

```
lemma Fulfilled_Requires_Release [reuse, use_induction,
    ↪ heuristic=S]:
    "All ptr x #i.
        FulfilledProduced(ptr, x) @ #i
     ==> Ex R #k. PreimageReleased(R, x) @ #k & #k < #i"
```

e) `Fulfilled_Source` (A): A fulfilment consumed by a settlement stems from an earlier redemption of that HTLC or from an earlier skip.

```
lemma Fulfilled_Source [reuse]:
    "All p x #i. UsedFulfilled(p, x) @ #i
     ==> (Ex #j. Redeemed(p, h(x)) @ #j & #j < #i)
       | (Ex #j. Skipped(p, x) @ #j & #j < #i)"
```

D. Atomicity (Group 4)

a) `Payment_Atomicity_Under_Liveness` (A): If two forwarders `F1` and `F2` forward consecutive HTLCs on the same hash y (the outgoing channel of `F1` is the incoming channel of `F2`), the HTLC on the outgoing channel of `F2` is redeemed, and no party's long-term key is ever compromised, then the HTLC on the incoming channel of `F1` is never refunded: in a run without any key compromise, once the payment is redeemed downstream of two forwarders, the party upstream of them cannot be refunded. The lemma is proved in the full model, i.e., under the liveness restrictions; it also verifies with T1 removed (Table II). It is Lemma 1.

```
lemma Payment_Atomicity_Under_Liveness:
    "All F1 F2 Prev1 Next1 Prev2 Next2 ptrSF1 ptrF1F2 ptrF2R y
     ↪ #i #j #k.
        HTLCForwarded(F1, Prev1, Next1, ptrSF1, ptrF1F2, y) @ #i
     & HTLCForwarded(F2, Prev2, Next2, ptrF1F2, ptrF2R, y) @ #j
     & Redeemed(ptrF2R, y) @ #k
     & (not Ex P #l. LtkCompromised(P) @ #l)
     ==> not (Ex Up #m. Refunded(Up, ptrSF1, y) @ #m)"
```

b) `Settle_Excludes_Sender_Refund` (A): If the sender has settled an HTLC with a preimage, then that same HTLC is never refunded: settlement and refund of the sender's HTLC are mutually exclusive.

```
lemma Settle_Excludes_Sender_Refund [heuristic={sender_pending}]:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> not (Ex #j. Refunded(S, ptr, h(x)) @ #j)"
```

c) `OneOutcomePerHTLC` (A): No HTLC is both redeemed and refunded. Both outcomes emit the action `Old` for the same channel and hash, which the restriction `OneResolutionPerHTLC` allows only once.

```
lemma OneOutcomePerHTLC [reuse]:
    "All Up ptr y #i #j.
       Redeemed(ptr, y) @ #i & Refunded(Up, ptr, y) @ #j ==> F"
```

d) `Multihop_Routing_Reachable` (E): Non-vacuity witness. The routing phase of a payment over two forwarders is reachable: the sender offers, both forwarders forward and the receiver fulfils, with no other offer or forward, at most one HTLC per channel, and no intermediary claim or sender settlement.

```
lemma Multihop_Routing_Reachable:
    exists-trace
    "Ex S F1 F2 R p0 p1 p2 x #t2 #t3 #t4 #t5.
        HTLCOffered(S, F1, p0, h(x)) @ #t2
```

```
      & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #t3
      & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #t4
      & HTLCFulfilled(R, F2, p2, h(x)) @ #t5
      & #t2 < #t3 & #t3 < #t4 & #t4 < #t5
      & not (Ex A B p y #u. HTLCOffered(A, B, p, y) @ #u & not
      ↪ (A = S & B = F1))
      & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev,
      ↪ Next, pi, po, y) @ #u
              & not (Me = F1 & Prev = S & Next = F2)
              & not (Me = F2 & Prev = F1 & Next = R))
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
      & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @
      ↪ #e)
      & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

e) `Sanity_Honest_Payment_Completes` (E): Non-vacuity witness. An honest payment over one forwarder completes end to end: the offer, the forward, the fulfilment, the forwarder's claim and the sender's settlement occur, with no key compromise, at most one HTLC per channel, exactly this claim and this settlement, and no refund.

```
lemma Sanity_Honest_Payment_Completes:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #c #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & IntermediaryClaimed(F1, S, p0, x) @ #c
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e
      ↪ ==> #e = #c)
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==>
      ↪ #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

f) `Sanity_Atomic_Payment_Completes` (E): Non-vacuity witness. An honest payment over two forwarders completes end to end: the offer, both forwards, the fulfilment, both intermediary claims and the sender's settlement occur, with no key compromise, at most one HTLC per channel, exactly these claims and this settlement, and no refund. It shows that the premise of `Payment_Atomicity_Under_Liveness` is satisfiable.

```
lemma Sanity_Atomic_Payment_Completes:
  exists-trace
  "Ex S F1 F2 R p0 p1 p2 x #o #a #b #r #c1 #c2 #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #a
    & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #b
    & HTLCFulfilled(R, F2, p2, h(x)) @ #r
    & IntermediaryClaimed(F2, F1, p1, x) @ #c2
    & IntermediaryClaimed(F1, S, p0, x) @ #c1
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e
      ↪ ==> (#e = #c1 | #e = #c2))
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==>
      ↪ #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

g) `Distinct_Parties_Configuration` (E): Non-vacuity witness. Three channels among four distinct parties can be opened with no compromise, update, publication, close, cheat or forward, and with at most one HTLC per channel.

```
lemma Distinct_Parties_Configuration:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R #t1 #t2 #t3.
       ChannelOpen(S, F1, ptrSF1) @ #t1
     & ChannelOpen(F1, F2, ptrF1F2) @ #t2
     & ChannelOpen(F2, R, ptrF2R) @ #t3
     & not (S = F1) & not (S = F2) & not (S = R)
     & not (F1 = F2) & not (F1 = R) & not (F2 = R)
     & not (Ex P #i. LtkCompromised(P) @ #i)
     & not (Ex P Q p n #j. StateUpdate(P, Q, p, n) @ #j)
     & not (Ex P n #k. PublishState(P, n) @ #k)
     & not (Ex P Q p n #l. Close(P, Q, p, n) @ #l)
     & not (Ex P n #m. Cheat(P, n) @ #m)
     & not (Ex Me Prev Next pi po y #n. HTLCForwarded(Me, Prev,
     ↪ Next, pi, po, y) @ #n)
     & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
     ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

E. Fees (Group 5)

Amounts are local to a hop. `Forward_HTLC` receives the incoming amount, the fee and the outgoing amount as inputs and emits an `Eq` action stating that the incoming amount is the outgoing amount plus the fee, which the `Equality` restriction enforces; on every hop, the incoming amount therefore equals the outgoing amount plus the fee by construction (Sec. V-A4). The offer, the invoice and the route carry no amount, so amounts are not chained along the path and the model makes no end-to-end value statement. The fee guarantee is the following lemma.

a) `Fee_Only_On_Successful_Forward` (A): A forwarder whose key is never compromised earns its fee only after its outgoing HTLC was redeemed and after it forwarded on that hop. It is Lemma 5.

```
lemma Fee_Only_On_Successful_Forward
      ↪ [heuristic={route_info_first}]:
    "All Me ptrIn ptrOut y %fee #i.
       FeeEarned(Me, ptrIn, ptrOut, y, %fee) @ #i
     & not (Ex #c. LtkCompromised(Me) @ #c)
    ==> (Ex #j. Redeemed(ptrOut, y) @ #j & #j < #i)
      & (Ex Prev Next #f. HTLCForwarded(Me, Prev, Next, ptrIn,
      ↪ ptrOut, y) @ #f & #f < #i)"
```

F. Refund and Griefing (Group 6)

a) `Refund_Requires_Timeout` (A): Every refund of an HTLC is preceded by the timeout of its channel: an HTLC is refunded only after that channel has timed out.

```
lemma Refund_Requires_Timeout [reuse]:
    "All Up ptr y #i.
       Refunded(Up, ptr, y) @ #i
     ==> Ex #j. TimedOut(ptr) @ #j & #j < #i"
```

b) `Loss_Requires_Inaction` (A): If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its incoming HTLC is never redeemed. An intermediary therefore loses funds only through its own inaction, by failing to redeem its incoming HTLC before it times out, and never because the same HTLC was both redeemed and refunded (the two outcomes are exclusive).

```
lemma Loss_Requires_Inaction:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
       HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
     & Redeemed(ptrOut, y) @ #j
     & Refunded(Up, ptrIn, y) @ #k
     ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```

c) `Refund_Possible` (E): Reachability witness. A payment can be refunded: the receiver issues an invoice, the sender offers an HTLC on its hash, and the sender's HTLC is refunded, with no settlement, no key compromise,

no forwarding, and at most one HTLC per channel. Since a refund requires a timeout (`Refund_Requires_Timeout`), the sender's funds stay locked until then; Sec. V-E reports this trace as reachable griefing of the sender.

```
lemma Refund_Possible:
    exists-trace
    "Ex S F1 R p y #i #j #k.
        InvoiceCreated(R, S, y) @ #j
      & HTLCOffered(S, F1, p, y) @ #k
      & Refunded(S, p, y) @ #i
      & #j < #k & #k < #i
      & not (Ex A MidP B p2 x #l. SenderSettled(A, MidP, B, p2,
      ↪ x) @ #l)
      & not (Ex P #r. LtkCompromised(P) @ #r)
      & not (Ex Me Prev Next pi po y2 #t1. HTLCForwarded(Me,
      ↪ Prev, Next, pi, po, y2) @ #t1)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

G. Wormhole (Group 7)

a) `Wormhole_Fee_Theft_Reachable` (E): Reachability witness (attack). There is a trace on the route Alice → Att1 → Indiana → Att2 → Bob in which Bob issues the invoice and redeems his incoming HTLC and Alice settles her HTLC, yet the outgoing HTLC of the intermediary Indiana is refunded and Indiana earns no fee. Alice, Bob and Indiana are never compromised, Att1 and Att2 are distinct parties, and every channel carries at most one HTLC. This is the wormhole [6] of Fig. 4: the preimage is relayed around Indiana by rule `AttackerSkip`, which only two compromised parties on the same payment can apply. TAMARIN does not find this trace automatically: `make verify` replays a saved 116-step proof. The statement, in the pretty-printed notation of TAMARIN, is shown in Fig. 5. In the PTLC model of `lightning_aux.spthy`, the corresponding wormhole around an honest hop is unreachable (`No_Wormhole_Around_Honest_Hop`), while the HTLC baseline reaches it (`Wormhole_Around_Honest_Hop`); see Appendix A-K.

b) `Skipped_Requires_Compromise` (A): Every wormhole skip happens after some party's long-term key was compromised (Lemma 6 in Sec. V-E); its statement is in Appendix B-F. It is also re-proved without the liveness restrictions (`make general`).

H. Intermediary Safety and Timing (Group 8)

1) Intermediary Safety:

a) `Intermediary_Never_Loses_Under_Liveness` (A): If an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its long-term key was compromised. Equivalently, an honest intermediary whose outgoing HTLC is redeemed never has its incoming HTLC refunded. The stronger lemma below places the compromise before the incoming timeout. This lemma is Lemma 2; it also verifies with T1 or T2 removed (Table II).

```
lemma Intermediary_Never_Loses_Under_Liveness:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l. LtkCompromised(P) @ #l"
```

b) `Intermediary_Never_Loses_Strong` (A): A forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded was compromised before its incoming HTLC timed out. It is Lemma 3; it also verifies with T1 or T2 removed (Table II).

```
lemma Intermediary_Never_Loses_Strong:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l #t. LtkCompromised(P) @ #l & TimedOut(ptrIn) @ #t
     ↪ & #l < #t"
```

c) `Sanity_Compromised_Intermediary_Can_Lose` (E): Non-vacuity witness for `Intermediary_Never_Loses_Under_Liveness`: a compromised forwarder can lose (its outgoing HTLC is fulfilled while its incoming HTLC is refunded), with at most one HTLC per channel, no intermediary claim and no sender settlement.

```
lemma Sanity_Compromised_Intermediary_Can_Lose:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u #k.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & LtkCompromised(F1) @ #k
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

d) `Necessity_IMC_Honest_Intermediary_Loses` (E): Checked with `-D=NO_IMC` and `-D=NECESSITY` (`make necessity`, 28 steps). Without T3 an honest forwarder can lose: there is a trace in which its outgoing HTLC is fulfilled by the receiver while its incoming HTLC is refunded to the sender, with no key compromise, no claim and no settlement. Hence T3 is necessary for intermediary safety (Sec. V-D).

```
lemma Necessity_IMC_Honest_Intermediary_Loses:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

2) Timing:

a) `Timeout_Race_Blocked` (A): No intermediary that is not compromised before its incoming timeout has its incoming HTLC refunded when that timeout happens strictly before its outgoing HTLC is redeemed. The lemma also verifies with any one of T1, T2 and T3 removed (4 steps each, Table II), so it does not depend on any single liveness restriction. In the abstract TIMEOUT model of `lightning_aux.spthy`, where either channel may time out at any point and the outgoing HTLC may be redeemed at any time, the race is reachable (`Early_Timeout_Race`, E, 6 steps; Appendix A-K).

```
lemma Timeout_Race_Blocked:
    "not (Ex P Prev Next Up ptrIn ptrOut y #l #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #l
```

```
lemma Wormhole_Fee_Theft_Reachable:
  exists-trace
  "∃ Alice Bob Att1 Att2 Indiana ptrAliceAtt1 ptrAtt1Indiana
     ptrIndianaAtt2 ptrAtt2Bob x #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
    ((((((((((((((InvoiceCreated( Bob, Alice, h(x) ) @ #t1) ∧
                  (HTLCOffered( Alice, Att1, ptrAliceAtt1, h(x) ) @ #t2)) ∧
                 (HTLCForwarded( Att1, Alice, Indiana, ptrAliceAtt1, ptrAtt1Indiana,
                                 h(x)
                  ) @ #t3)) ∧
                (HTLCForwarded( Indiana, Att1, Att2, ptrAtt1Indiana,
                                ptrIndianaAtt2, h(x)
                 ) @ #t4)) ∧
               (HTLCForwarded( Att2, Indiana, Bob, ptrIndianaAtt2, ptrAtt2Bob,
                               h(x)
                ) @ #t5)) ∧
              (HTLCFulfilled( Bob, Att2, ptrAtt2Bob, h(x) ) @ #t6)) ∧
             (SenderSettled( Alice, Att1, Bob, ptrAliceAtt1, x ) @ #t7)) ∧
            (Refunded( Indiana, ptrIndianaAtt2, h(x) ) @ #t8)) ∧
           (¬(Indiana = Alice))) ∧
          (¬(Indiana = Bob))) ∧
         (¬(Att1 = Att2))) ∧
        (¬(∃ #c. LtkCompromised( Alice ) @ #c))) ∧
       (¬(∃ #c. LtkCompromised( Bob ) @ #c))) ∧
      (¬(∃ #c. LtkCompromised( Indiana ) @ #c))) ∧
     (¬(∃ ptr1 ptr2 yy %fee #k.
         FeeEarned( Indiana, ptr1, ptr2, yy, %fee ) @ #k))) ∧
    (¬(∃ ptr y1 y2 #m #n.
        ((HTLCAdded( ptr, y1 ) @ #m) ∧ (HTLCAdded( ptr, y2 ) @ #n)) ∧
        (¬(#m = #n))))"
```

Fig. 5. Statement of `Wormhole_Fee_Theft_Reachable` in `lightning.spthy`, in the pretty-printed notation of Tamarin (exists-trace; verified by replaying a saved 116-step proof).

```
      & TimedOut(ptrIn)     @ #k
      & Refunded(Up, ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k))"
```

b) `ForwardTimeGap` (A): If both channels of a forward time out and the forwarder is not compromised before its incoming timeout, the outgoing HTLC times out strictly before the incoming one. The order of a hop's two timeouts is thus not assumed: it is this lemma, derived from T2 (Sec. IV-B). It also verifies with T1 or T3 removed (Table II). With `-D=NECESSITY`, the same statement is declared without `[reuse]`, so that no other proof uses it in the necessity runs and in the general mode.

```
lemma ForwardTimeGap [reuse]:
    "All P Prev Next ptrIn ptrOut y #f #j #k.
       HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrOut) @ #j
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> #j < #k"
```

I. Hop Bypass (Group 9)

If the payment settles at the sender while a forwarder's own hop is refunded (its outgoing HTLC is refunded, `RefundHop`), some party's key was compromised. The conclusion does not say which party, or when. The lemmas for positions 1 to 6 are proved by `make verify`, in the full model under T1 to T3, and need no single-offer premise; `Any_Hop_Skip_Requires_Compromise` covers any position on a route of any length under a single-offer premise (every offer on $h(x)$ uses the sender's channel `pSF`) and is proved by `make general`. Sec. VI discusses these results.

1) Positions 1 to 6:

a) `First_Hop_Skip_Requires_Compromise` (A): The first forwarder cannot be bypassed without a compromise: if the sender settles its HTLC on channel `pSF` while the hop of its first forwarder `F1` from `pSF` is refunded, some party's key was compromised.

```
lemma First_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
    "All S F1 R pSF Next pOut x #s #f #r.
       SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, Next, pSF, pOut, h(x)) @ #f
      & RefundHop(pSF, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

b) `Second_Hop_Skip_Requires_Compromise` (A): The second forwarder (the position of Indiana in `Wormhole_Fee_Theft_Reachable`) cannot be bypassed without a compromise.

```
lemma Second_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
    "All S F1 R pSF x F2 p1 Next pOut #s #f1 #f2 #r.
       SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, Next, p1, pOut, h(x)) @ #f2
      & RefundHop(p1, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

c) `Hop3_Skip_Requires_Compromise` (A): The forwarder at position 3 cannot be bypassed without a compromise. The statement adds a third forward to the chain of `Second_Hop_Skip_Requires_Compromise`; the statements for positions 4 to 6 each add one more.

```
lemma Hop3_Skip_Requires_Compromise [heuristic={lemmas_first}]:
```

```
    "All S R x pSF pOut Next F1 F2 F3 p1 p2 #s #r #f1 #f2 #f3.
       SenderSettled(S, F1, R, pSF, x) @ #s
     & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
     & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
     & HTLCForwarded(F3, F2, Next, p2, pOut, h(x)) @ #f3
     & RefundHop(p2, pOut, h(x)) @ #r
    ==> Ex P #c. LtkCompromised(P) @ #c"
```

d) `Hop4_Skip_Requires_Compromise` (A): The forwarder at position 4 cannot be bypassed without a compromise.

```
lemma Hop4_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 p1 p2 p3 #s #r #f1 #f2
     ↪ #f3 #f4.
       SenderSettled(S, F1, R, pSF, x) @ #s
     & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
     & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
     & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
     & HTLCForwarded(F4, F3, Next, p3, pOut, h(x)) @ #f4
     & RefundHop(p3, pOut, h(x)) @ #r
    ==> Ex P #c. LtkCompromised(P) @ #c"
```

e) `Hop5_Skip_Requires_Compromise` (A): The forwarder at position 5 cannot be bypassed without a compromise.

```
lemma Hop5_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 p1 p2 p3 p4 #s #r
     ↪ #f1 #f2 #f3 #f4 #f5.
       SenderSettled(S, F1, R, pSF, x) @ #s
     & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
     & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
     & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
     & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
     & HTLCForwarded(F5, F4, Next, p4, pOut, h(x)) @ #f5
     & RefundHop(p4, pOut, h(x)) @ #r
    ==> Ex P #c. LtkCompromised(P) @ #c"
```

f) `Hop6_Skip_Requires_Compromise` (A): The forwarder at position 6 cannot be bypassed without a compromise.

```
lemma Hop6_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 F6 p1 p2 p3 p4 p5 #s
     ↪ #r #f1 #f2 #f3 #f4 #f5 #f6.
       SenderSettled(S, F1, R, pSF, x) @ #s
     & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
     & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
     & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
     & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
     & HTLCForwarded(F5, F4, F6, p4, p5, h(x)) @ #f5
     & HTLCForwarded(F6, F5, Next, p5, pOut, h(x)) @ #f6
     & RefundHop(p5, pOut, h(x)) @ #r
    ==> Ex P #c. LtkCompromised(P) @ #c"
```

2) Helper Lemmas: The hop-bypass lemmas rest on seven helper lemmas, all of kind A and declared `[reuse]`. All seven are also re-proved in `make general`.

a) `Forward_Unique_Per_Incoming` (A): An incoming HTLC is forwarded at most once: two forwards of the same hash from the same incoming channel are the same event.

```
lemma Forward_Unique_Per_Incoming [reuse,
      ↪ heuristic={route_first}]:
    "All M1 P1 N1 M2 P2 N2 pIn o1 o2 y #i #j.
       HTLCForwarded(M1, P1, N1, pIn, o1, y) @ #i
     & HTLCForwarded(M2, P2, N2, pIn, o2, y) @ #j
    ==> #i = #j"
```

b) `SettledHop_From_Forward` (A): A hop settles only if it was forwarded earlier, from the same incoming to the same outgoing channel.

```
lemma SettledHop_From_Forward [reuse, heuristic={route_first}]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
    ==> Ex M P N #j. HTLCForwarded(M, P, N, pIn, pOut, y) @ #j
     ↪ & #j < #i"
```

c) `No_Settle_And_Refund_Same_Hop` (A): No hop both settles and has its outgoing HTLC refunded.

```
lemma No_Settle_And_Refund_Same_Hop [reuse,
      ↪ heuristic={route_first}]:
    "All pIn pOut y #i #j. SettledHop(pIn, pOut, y) @ #i &
     ↪ RefundHop(pIn, pOut, y) @ #j ==> F"
```

d) `No_Fulfill_On_Forwarded_Incoming` (A): The HTLC on a channel is never both fulfilled by a receiver and forwarded onwards.

```
lemma No_Fulfill_On_Forwarded_Incoming [reuse,
      ↪ heuristic={route_first}]:
    "All R Q M P N p o y #i #j.
       HTLCFulfilled(R, Q, p, y) @ #i & HTLCForwarded(M, P, N,
     ↪ p, o, y) @ #j ==> F"
```

e) `Redeemed_Source` (A): An HTLC is redeemed only by the settlement of the hop for which it is the incoming HTLC, or by the receiver's fulfilment, in the same step.

```
lemma Redeemed_Source [reuse]:
    "All p y #i. Redeemed(p, y) @ #i
     ==> (Ex o. SettledHop(p, o, y) @ #i)
       | (Ex R Q. HTLCFulfilled(R, Q, p, y) @ #i)"
```

f) `SenderSettled_Uses_Fulfilled` (A): A sender settlement consumes the fulfilment of the sender's own channel.

```
lemma SenderSettled_Uses_Fulfilled [reuse]:
    "All S F1 R p x #i. SenderSettled(S, F1, R, p, x) @ #i ==>
     ↪ UsedFulfilled(p, x) @ #i"
```

g) `SettledHop_Uses_Outgoing` (A): A hop settlement consumes the fulfilment of the hop's outgoing channel, with a preimage of the hop's hash.

```
lemma SettledHop_Uses_Outgoing [reuse]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
     ==> Ex x. UsedFulfilled(pOut, x) @ #i & h(x) = y"
```

3) Routes of Any Length (General Mode):

a) `Any_Hop_Skip_Requires_Compromise` (A): Proved by `make general` in 7 steps, with the flags shown at the start of this appendix. A single-offer payment that settles while any forwarder's own hop is refunded requires a compromise. The lemma is proved in the model without the three liveness restrictions; removing restrictions only adds traces, so the statement also holds in the full model. It is Theorem 2; the statement is written in the pretty-printed notation of Tamarin.

```
lemma Any_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
       (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f))
     ↪ ∧
      (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p =
     ↪ pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

The general theorem uses a chain of seven helper lemmas, all of kind A and proved only in this mode; we list them by name.

- `Forward_Parent`: the statement of `Forward_Requires_Incoming`, declared `[reuse]` for this mode.

- `Hop_Redeem_Propagates`: with no compromise, if the incoming HTLC of a forwarder is redeemed, so is its outgoing HTLC.
- `Sender_Channel_Redeemed`: with no compromise, a sender settlement implies that the sender's channel was redeemed.
- `Root_First`: every forward either forwards an offered HTLC or is preceded by a forward that does.
- `Next_Is_Child`: with a single offer, every forward on the hash that follows a forward to `Me` is, or comes after, the forward of `Me` from that channel.
- `Route_Redeemed_Gen`: with a single offer and no compromise, if the incoming HTLC of a forward is redeemed, so is the incoming HTLC of every forward on the same hash at the same time or later.
- `Route_Redeemed_No_Compromise`: with a single offer and no compromise, if the sender settles, the incoming HTLC of every forwarder on the hash is redeemed.

`make general` proves 24 lemmas: the theorem, these seven helpers, and 16 lemmas of `make verify` re-proved in the same mode, namely the seven helper lemmas of the hop-bypass family above, `OneOutcomePerHTLC`, `Ltk_Known_Implies_Compromised`, `Invoice_Released_Once`, `Invoice_Has_Secret_Preimage`, `HTLC_On_Opened_Channel`, `Fulfilled_Requires_Release`, `Refund_Requires_Timeout`, `Fulfilled_Source` and `Skipped_Requires_Compromise`.

J. Channel Layer and Keys (Group 0)

These lemmas concern the channel layer (Sec. V-A1) and the long-term keys. Their action facts read as follows.

- `StateUpdate(A, B, ptr, n)`: the channel moved to state `n` after both parties revoked their previous state.
- `NewStateBuilt(A, B, ptr, n, fA, fB)`: state `n` with balances `fA` and `fB` was built, at opening or by `Update_State`.
- `FundsLocked(A, B, ptr)`: the funds of both parties were locked when the channel opened.
- `PublishState(A, n)`: `A` published its commitment of state `n`.
- `Cheat(A, n)` and `Punish(B, A, n)`: `A` published its revoked state `n`, and `B` punished this and took all funds of the channel.
- `Close(A, B, ptr, n)`: the channel was closed cooperatively in state `n`.
- `OnChainDelayedProduce(A, n)` and `OnChainInstantProduce(A, n)`: an on-chain transaction of state `n` created a delayed (CSV-locked) or an instant output for `A`.
- `ClaimFunds(A, n)` and `FundsSettled(A, n)`: `A` claimed its delayed output after the CSV delay, or received its instant output.
- `SettledTx(A, tx)`: the transaction `tx` paid `A`.

a) `state_update` (A): A channel is updated only after the same two parties opened it.

```
lemma state_update [use_induction]:
    "All A B ptr n #i.
       StateUpdate(A, B, ptr, n) @ #i
     ==> Ex #j. ChannelOpen(A, B, ptr) @ #j & #j < #i"
```

b) `Funds_Locked_Before_Update` (A): A channel is updated only after the funds of both parties were locked in it.

```
lemma Funds_Locked_Before_Update [use_induction]:
    "All A B ptr n #i.
       StateUpdate(A, B, ptr, n) @ #i
     ==> Ex #j. FundsLocked(A, B, ptr) @ #j & #j < #i"
```

c) `Update_Requires_Negotiation` (A): A channel moves to a new state only after both parties built that state, with its balances.

```
lemma Update_Requires_Negotiation [use_induction]:
    "All A B ptr n #i.
         StateUpdate(A, B, ptr, n) @ #i
     ==> Ex fA fB #j. NewStateBuilt(A, B, ptr, n, fA, fB) @ #j &
      ↪ #j < #i"
```

d) `delayed_funds` (A): A party claims a delayed output of state `n` only after an on-chain transaction of state `n` created that output for it.

```
lemma delayed_funds:
    "All A n #i.
       ClaimFunds(A, n) @ #i
     ==> Ex #j. OnChainDelayedProduce(A, n) @ #j & #j < #i"
```

e) `instant_funds` (A): A party receives an instant output of state `n` only after an on-chain transaction of state `n` created that output for it.

```
lemma instant_funds:
    "All A n #i.
       FundsSettled(A, n) @ #i
     ==> Ex #j. OnChainInstantProduce(A, n) @ #j & #j < #i"
```

f) `settlement_is_traceable` (A): Every instant settlement names the transaction that pays the party, in the same step.

```
lemma settlement_is_traceable:
    "All A n #i.
       FundsSettled(A, n) @ #i
     ==> Ex tx #j. SettledTx(A, tx) @ #j & #j = #i"
```

g) `No_Punishment_Without_Cheating` (A): A party is punished for state `n` only if it published that revoked state earlier, or if its long-term key was compromised earlier.

```
lemma No_Punishment_Without_Cheating:
    "All A B n #i.
        Punish(B, A, n) @ #i
     ==> (Ex #k. Cheat(A, n) @ #k & #k < #i)
       | (Ex #j. LtkCompromised(A) @ #j & #j < #i)"
```

h) `Ltk_Known_Implies_Compromised` (A): The adversary knows a long-term key only if that key was compromised. Here `!KU(ltk)` means that the adversary can construct `ltk`.

```
lemma Ltk_Known_Implies_Compromised [reuse]:
    "All A ltk #i #j.
        KeyGen(A, ltk) @ #i
      & !KU(ltk) @ #j
     ==> Ex #k. LtkCompromised(A) @ #k"
```

i) `Protocol_execution` (E): Non-vacuity witness. A channel can be opened, a new state built and committed

by an update, and the channel closed cooperatively in that state, in this order, with at most one HTLC per channel.

```
lemma Protocol_execution:
    exists-trace
    "Ex A B ptr n fA fB #i #j #k #l.
        ChannelOpen(A, B, ptr) @ #i
      & NewStateBuilt(A, B, ptr, n, fA, fB) @ #j
      & StateUpdate(A, B, ptr, n) @ #k
      & Close(A, B, ptr, n) @ #l
      & #i < #j & #j < #k & #k < #l
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
     ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

j) `Cooperative_Close_Execution` (E): Non-vacuity witness. A channel can be opened and closed cooperatively without any state update, with at most one HTLC per channel.

```
lemma Cooperative_Close_Execution:
    exists-trace
    "Ex A B ptr n #k #i.
        ChannelOpen(A, B, ptr) @ #k & Close(A, B, ptr, n) @ #i &
     ↪ #k < #i
      & not (Ex A2 B2 ptr2 n2 #j. StateUpdate(A2, B2, ptr2, n2)
     ↪ @ #j)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
     ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

K. Auxiliary Models

The auxiliary models of `lightning_aux.spthy` are self-contained and are not composed with `lightning.spthy` (Sec. IV-C). Table III lists their 21 lemmas, all verified by `make aux`; we print the key statements below.

The PTLC model (`-D=PTLC`) has the fixed route S → H1 → H2 → H3 → R over the channels `c0` to `c3`. Each channel has its own secret, derived from the secret of the next channel by the hash chain $s_{in} = h(\langle s_{out}, t\rangle)$ with a fresh tweak t per hop, a symbolic stand-in for the adaptor-signature points of [6]; locks are hashes of the secrets. The HTLC baseline (`-D=HTLC`) has the same route with one secret x for every channel. In both models a compromised hop leaks its tweak, the secret used to claim a channel becomes public, and a compromised party can claim its incoming channel with any value the adversary knows. There is no skip rule: a wormhole must be built from the adversary's knowledge. The CLOCK model (`-D=CLOCK`) gives channels concrete CLTV deadlines on a block clock, lets a forwarder forward only if its outgoing deadline is strictly lower than its incoming one, and records the block heights at which HTLCs are redeemed, claimed and refunded. The CLTV model (`-D=CLTV`) registers hops whose incoming deadline exceeds the outgoing one by a positive delta (restriction `DeltaPositive`) and reveals a preimage only strictly before the outgoing deadline (restriction `FulfillInTime`). The TIMEOUT model (`-D=TIMEOUT`) has four rules and no ordering: either channel may time out at any point, and the outgoing HTLC may be redeemed at any time.

a) `No_Wormhole_Around_Honest_Hop` (A, PTLC): In the PTLC model, the channel `c0` of the sender is never redeemed while the outgoing channel `c2` of an uncompromised H2 is refunded, unless the sender is compromised. The result depends on the tweak of H2 staying secret: once H2 is compromised, the wormhole is reachable (`Control_Wormhole_If_H2_Compromised`), and the refund of the outgoing channel of an uncompromised H2 alone is reachable too (`Control_Refund_Of_H2_Outgoing`).

TABLE III
LEMMAS OF THE AUXILIARY MODELS (`LIGHTNING_AUX.SPTHY`, `MAKE AUX`, 21/21 VERIFIED). A: ALL-TRACES; E: EXISTS-TRACE; STEPS: SIZE OF THE PROOF FOUND BY TAMARIN.

Flag	Lemma	Kind	Steps
PTLC	Honest_Payment_Completes	E	10
	Colluders_Present_Payment_Completes	E	14
	No_Wormhole_Around_Honest_Hop	A	25
	Control_Refund_Of_H2_Outgoing	E	6
	Control_Wormhole_If_H2_Compromised	E	18
HTLC	Honest_Payment_Completes	E	10
	Colluders_Present_Payment_Completes	E	8
	Wormhole_Around_Honest_Hop	E	12
CLOCK	HTLC_Needs_Open_Channel	A	36
	No_HTLC_After_Close	A	367
	Claim_Window_Exists	A	79
	Transitive_Preimage_Before_Upstream_Deadline	A	187
	Loss_Implies_Skipped_Claim	A	861
	Outcome_Exclusive	A	2
	Redeem_Reachable	E	7
	Refund_Reachable	E	7
	Honest_Flow_Possible	E	14
CLTV	CLTV_Gap_Is_Positive	A	2
	Claim_Window_Nonempty	A	2
	Staggered_Path_Safe	A	2
TIMEOUT	Early_Timeout_Race	E	6

```
lemma No_Wormhole_Around_Honest_Hop:
  all-traces
  "not (
Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k))"
```

b) `Wormhole_Around_Honest_Hop` (E, HTLC): With HTLCs, that wormhole around an uncompromised H2 is reachable, with an uncompromised sender.

```
lemma Wormhole_Around_Honest_Hop:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k)"
```

c) `Claim_Window_Exists` (A, CLOCK): If a forwarder's outgoing HTLC is redeemed at block height `bOut` and its incoming HTLC is refunded at height `bRef`, then `bOut` is strictly lower than `bRef`: the forwarder had a non-empty window in which to claim.

```
lemma Claim_Window_Exists:
    "All F pIn pOut dIn dOut y bOut bRef #f #r #u.
        Forwarded(F, pIn, pOut, dIn, dOut, y) @ #f
      & RedeemedAt(pOut, y, bOut) @ #r
      & RefundedAt(pIn,  y, bRef) @ #u
```

```
    ==> bOut << bRef"
```

d) `Loss_Implies_Skipped_Claim` (A, CLOCK): A forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded never claimed its incoming HTLC.

```
lemma Loss_Implies_Skipped_Claim:
    "All F pIn pOut dIn dOut y #f #r #u.
       Forwarded(F, pIn, pOut, dIn, dOut, y) @ #f
     & Redeemed(pOut, y) @ #r
     & Refunded(pIn,  y) @ #u
     ==> not (Ex b #c. ClaimedAt(pIn, y, b) @ #c)"
```

e) `Claim_Window_Nonempty` (A, CLTV): A preimage revealed before the outgoing deadline of a hop is revealed before its incoming deadline.

```
lemma Claim_Window_Nonempty [reuse]:
    "All ptrOut ptrIn d_out d_in b #i #j.
       HopRegistered(ptrOut, ptrIn, d_out, d_in) @ #i
     & Revealed(b, ptrOut, d_out) @ #j
     ==> b << d_in"
```

f) `Staggered_Path_Safe` (A, CLTV): Over two staggered hops, a preimage revealed before the tightest deadline is revealed before both upstream deadlines, so both intermediaries have a non-empty claim window.

```
lemma Staggered_Path_Safe:
    "All ptr1 ptr2 ptr2b ptr3 d1 d2 d3 b #i #j #k.
       HopRegistered(ptr1, ptr2, d1, d2) @ #i
     & HopRegistered(ptr2b, ptr3, d2, d3) @ #j
     & Revealed(b, ptr1, d1) @ #k
     ==> b << d2 & b << d3"
```

g) `Early_Timeout_Race` (E, TIMEOUT): Without any ordering of timeouts, a forwarder's incoming HTLC times out and is refunded, and its outgoing HTLC is redeemed after that timeout. `Timeout_Race_Blocked` excludes this pattern in `lightning.spthy` for forwarders that are not compromised before their incoming timeout.

```
lemma Early_Timeout_Race:
    exists-trace
    "Ex P ptrIn ptrOut y #f #k #j #i.
       ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
     & TimedOut(ptrIn)     @ #k
     & Refunded(ptrIn, y)  @ #j
     & Redeemed(ptrOut, y) @ #i
     & #k < #i"
```

Appendix B
Model Listings

This appendix collects the rules and lemma statements of `lightning.spthy` and `lightning_aux.spthy` that the main text describes in prose.

A. Structural Restrictions

Besides the liveness restrictions T1 to T3 (Sec. IV-B), `lightning.spthy` has six structural restrictions. `OneResolutionPerHTLC` is the interface assumption between the channel layer and the routing layer (Sec. IV): every rule that redeems or refunds an HTLC records the action fact `Old`, and each HTLC is resolved at most once. `OneHTLCPerHashPerChannel` lets a payment hash be added at most once per channel. `Equality` and `Inequality` turn the action facts `Eq` and `Neq` into constraints, which encodes checks such as signature verification. `OneTimeoutPerPtr` and `OneChannelPerPtr` let a channel time out and open at most once. The order of a hop's two timeouts is not a restriction: it is the lemma `ForwardTimeGap`, derived from T2 and `OneTimeoutPerPtr` and printed in Appendix A-H.

```
restriction OneResolutionPerHTLC:
    "All ptr y #i #j. Old(ptr, y) @ #i & Old(ptr, y) @ #j ==> #i
     ↪ = #j"

restriction OneHTLCPerHashPerChannel:
    "All ptr y #m #n. HTLCAdded(ptr, y) @ #m & HTLCAdded(ptr, y)
     ↪ @ #n ==> #m = #n"

restriction Inequality:
    "All x y #i. Neq(x, y) @ #i ==> not(x = y)"

restriction Equality:
    "All x y #i. Eq(x, y) @ #i ==> x = y"

restriction OneTimeoutPerPtr:
    "All ptr #i #j. TimedOut(ptr) @ #i & TimedOut(ptr) @ #j ==>
     ↪ #i = #j"

restriction OneChannelPerPtr:
    "All A B C D ptr #i #j.
       ChannelOpen(A, B, ptr) @ #i
     & ChannelOpen(C, D, ptr) @ #j
     ==> #i = #j"
```

B. Channel Opening and Update

These are the opening rule and the two rules of the update dance of Sec. V-A1. `Lock_Funds_And_Open` creates the two mirrors `StateA` and `StateB` of state `'0'`, the persistent facts that the other rules check, and the per-channel slot `Mutex(ptr)` (Sec. V-B).

```
rule Lock_Funds_And_Open:
    [ ReadyToLock($A, $B, ptr, sA0, sB0, commitA0, commitB0),
      Wallet($A, ~fA), Wallet($B, ~fB) ]
    --[ ChannelOpen($A, $B, ptr), FundsLocked($A, $B, ptr),
        NewStateBuilt($A, $B, ptr, '0', ~fA, ~fB) ]->
    [ StateA($A, $B, ptr, '0', sA0, commitA0, ~fA, ~fB),
      StateB($A, $B, ptr, '0', sB0, commitB0, ~fA, ~fB),
      !ValidOnChain(ptr),
      !ChannelRecord(ptr, $A, $B),
      !ChannelConnect(ptr, $A, $B),
      !ChannelConnect(ptr, $B, $A),
      Mutex(ptr) ]
```

`Update_State` commits a new state: it samples the next state index and both fresh revocation secrets, reads the two current mirrors and the new balances from the environment, and produces two pending mirrors, each keeping the old state next to the new, signed one.

```
rule Update_State:
    let newCommitA = <'commit', ptr, $A, $B, ~n_new, h(~sAnew)>
        newCommitB = <'commit', ptr, $B, $A, ~n_new, h(~sBnew)>
        sigAnew    = sign(newCommitA, ltkA)
        sigBnew    = sign(newCommitB, ltkB)
    in
    [ Fr(~n_new), Fr(~sAnew), Fr(~sBnew),
      !Ltk($A, ltkA), !Ltk($B, ltkB),
      StateA($A, $B, ptr, n_old, sAold, oldCommitA, fA, fB),
      StateB($A, $B, ptr, n_old, sBold, oldCommitB, fA, fB),
      In(<fA_new, fB_new>) ]
    --[ NewStateBuilt($A, $B, ptr, ~n_new, fA_new, fB_new) ]->
    [ PendingA($A, $B, ptr, n_old, sAold, oldCommitA,
               ~n_new, ~sAnew, newCommitA, sigAnew, fA_new,
     ↪ fB_new),
      PendingB($A, $B, ptr, n_old, sBold, oldCommitB,
               ~n_new, ~sBnew, newCommitB, sigBnew, fA_new,
     ↪ fB_new) ]
```

Only afterwards does `Revoke_Old_Secret` supersede the old state: it consumes the two pending mirrors, installs them as the new state, and records, for both

parties, the old revocation secret as the persistent `!RevokedSecret` and the old commitment as the linear token `RevokedCommitmentUnspent`. The signature checks of Sec. II-A become the two `Eq` action facts: a TAMARIN rule cannot fail, so restriction `Equality` turns these action facts into hard prerequisites. The order (commit, then revoke) is what protects a party that aborts mid-update: the old state is never revoked until a fully signed new one exists.

```
rule Revoke_Old_Secret:
    [ PendingA($A, $B, ptr, n_old, sAold, oldCommitA,
               n_new, sAnew, newCommitA, sigAnew, fA_new,
      ↪ fB_new),
      PendingB($A, $B, ptr, n_old, sBold, oldCommitB,
               n_new, sBnew, newCommitB, sigBnew, fA_new,
      ↪ fB_new),
      !Pk($A, pkA), !Pk($B, pkB) ]
    --[ StateUpdate($A, $B, ptr, n_new),
        Revoke($A, n_old, sAold),
        Revoke($B, n_old, sBold),
        Eq(verify(sigAnew, newCommitA, pkA), true),
        Eq(verify(sigBnew, newCommitB, pkB), true) ]->
    [ StateA($A, $B, ptr, n_new, sAnew, newCommitA, fA_new,
      ↪ fB_new),
      StateB($A, $B, ptr, n_new, sBnew, newCommitB, fA_new,
      ↪ fB_new),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      !RevokedSecret($B, n_old, sBold, oldCommitB),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA),
      RevokedCommitmentUnspent(ptr, $B, $A, n_old, oldCommitB) ]
```

C. Channel Closing and Punishment

The closing and punishment rules of Sec. V-A1; the rules for party B (`Publish_Current_B`, `Claim_Funds_B`, `Publish_Revoked_B`, `Punish_B`) are symmetric. A unilateral close spends the current mirror: `Publish_Current_A` turns `StateA` into an `OnChainDelayed` and an `OnChainInstant` output, and `Claim_Funds_A` later releases the delayed output to its owner; its `CSVPassed` action captures that time has passed.

```
rule Publish_Current_A:
    [ StateA($A, $B, ptr, n, sA, commitA, fA, fB),
      ↪ !ValidOnChain(ptr) ]
    --[ Publish($A, n, commitA),
        PublishState($A, n),
        OnChainDelayedProduce($A, n),
        OnChainInstantProduce($B, n) ]->
    [ OnChainDelayed($A, n, commitA),
      OnChainInstant($B, n, commitA) ]

rule Claim_Funds_A:
    [ OnChainDelayed($A, n, txContent) ]
    --[ ClaimFunds($A, n), CSVPassed($A, n), SettledTx($A,
      ↪ txContent) ]->
    [ FundsTo($A, n) ]
```

A cooperative close signs one shared settlement transaction instead.

```
rule Honest_Close:
    let closetx = <'commit', ptr, $A, $B, n, fA, fB>
        signA   = sign(closetx, ltkA)
        signB   = sign(closetx, ltkB)
    in
    [ StateA($A, $B, ptr, n, sA, commitA, fA, fB),
      StateB($A, $B, ptr, n, sB, commitB, fA, fB),
      !ValidOnChain(ptr),
      !Ltk($A, ltkA), !Ltk($B, ltkB),
      !Pk($A, pkA),   !Pk($B, pkB) ]
    --[ Close($A, $B, ptr, n),
        OnChainInstantProduce($A, n),
        OnChainInstantProduce($B, n),
        Eq(verify(signA, closetx, pkA), true),
        Eq(verify(signB, closetx, pkB), true) ]->
    [ OnChainInstant($A, n, closetx),
      OnChainInstant($B, n, closetx) ]
```

Punishment publishes a revoked state, which consumes the linear token `RevokedCommitmentUnspent` (the single-spend fix of Sec. V-B: our first version of `Publish_Revoked_A` had all of the premises below except this one), and then transfers the whole channel to the honest party. Persistent knowledge of the revocation secret is retained via `!RevokedSecret`; only spending the funds is linear.

```
rule Publish_Revoked_A:
    [ !ChannelRecord(ptr, $A, $B),
      !ValidOnChain(ptr),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA) ]
    --[ PublishRevoked($A, n_old, oldCommitA),
        Cheat($A, n_old) ]->
    [ OnChainCheat($A, $B, n_old, oldCommitA),
      RevealedSecret($A, n_old, sAold, oldCommitA) ]

rule Punish_A:
    [ OnChainCheat($A, $B, n_old, oldCommitA),
      RevealedSecret($A, n_old, sAold, oldCommitA) ]
    --[ Punish($B, $A, n_old) ]->
    [ AllFundsTo($B) ]
```

D. Invoice, Settlement and Refund

The routing rules of Sec. V-A2 besides `Forward_HTLC` and `AttackerSkip`, which are printed there. The invoice is tagged and names the receiver, the sender and the payment hash; the sender checks the receiver's signature before it offers the first HTLC.

```
rule Receiver_Create_Invoice:
    let y = h(~x)
        invoice = <'invoice', $Receiver, $Sender, y>
        sigR    = sign(invoice, ltkR)
    in
    [ Fr(~x), !Ltk($Receiver, ltkR) ]
    --[ InvoiceCreated($Receiver, $Sender, y),
        SecretPreimage($Receiver, ~x) ]->
    [ ReceiverInv($Receiver, $Sender, ~x),
      Out(<'SecretRS', invoice, sigR>) ]

rule Sender_Offer_HTLC:
    let invoice = <'invoice', $Receiver, $Sender, y>
    in
    [ In(<'SecretRS', invoice, sigR>),
      !Pk($Receiver, pkR),
      !Ltk($Sender, ltkS),
      !ChannelConnect(ptrSF, $Sender, $Forward1),
      Mutex(ptrSF) ]
    --[ HTLCOffered($Sender, $Forward1, ptrSF, y),
        OfferFor($Sender, $Receiver, ptrSF, y),
        HTLCAdded(ptrSF, y),
        Neq($Sender, $Forward1),
        Neq($Sender, $Receiver),
        Neq($Forward1, $Receiver),
        Eq(verify(sigR, invoice, pkR), true) ]->
    [ SenderPending($Sender, $Forward1, $Receiver, ptrSF, y),
      Route($Sender, $Forward1, ptrSF, y),
      !RouteInfo($Sender, $Forward1, ptrSF, y) ]
```

The settle round runs backwards on `Fulfilled` facts. The receiver's and the intermediary's settlement give the slot of the incoming channel back.

```
rule Receiver_Fulfill_HTLC:
    [ Route($Prev, $Receiver, ptrIn, y),
      ReceiverInv($Receiver, $Sender, x) ]
    --[ HTLCFulfilled($Receiver, $Prev, ptrIn, y),
        PreimageReleased($Receiver, x),
        Redeemed(ptrIn, y),
        Old(ptrIn, y),
        Eq(h(x), y) ]->
```

```
    [ Fulfilled(ptrIn, x), Mutex(ptrIn) ]

rule Intermediary_Settle:
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y,
      ↪ %vIn, %vOut, %fee),
      Fulfilled(ptrOut, x) ]
    --[ IntermediaryClaimed($Me, $Prev, ptrIn, x),
        UsedFulfilled(ptrOut, x),
        SettledHop(ptrIn, ptrOut, y),
        FeeEarned($Me, ptrIn, ptrOut, y, %fee),
        Redeemed(ptrIn, y),
        Old(ptrIn, y),
        FulfilledProduced(ptrIn, x),
        Eq(h(x), y) ]->
    [ Fulfilled(ptrIn, x), Mutex(ptrIn) ]

rule Sender_Settle:
    [ SenderPending($Sender, $Forward1, $Receiver, ptrSF, y),
      Fulfilled(ptrSF, x) ]
    --[ SenderSettled($Sender, $Forward1, $Receiver, ptrSF, x),
        UsedFulfilled(ptrSF, x),
        Eq(h(x), y) ]->
    [ ]
```

A timeout is an untimed event; after it, the pending HTLC on that channel is refunded to the party that offered it, and the slot is given back.

```
rule Open_Timeout:
    [ !ChannelConnect(ptr, A, B) ] --[ TimedOut(ptr) ]-> [
      ↪ !Timeout(ptr) ]

rule Sender_Timeout_Refund:
    [ SenderPending($Sender, $Forward1, $Receiver, ptrSF, y),
      ↪ !Timeout(ptrSF) ]
    --[ Refunded($Sender, ptrSF, y),
        Old(ptrSF, y) ]->
    [ Mutex(ptrSF) ]

rule Intermediary_Timeout_Refund:
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y,
      ↪ %vIn, %vOut, %fee),
      !Timeout(ptrOut) ]
    --[ Refunded($Me, ptrOut, y),
        RefundHop(ptrIn, ptrOut, y),
        Old(ptrOut, y) ]->
    [ Mutex(ptrOut) ]
```

E. Lemma Groups

Table IV sorts all lemmas of `lightning.spthy` into the groups of Sec. V-C, one per stage of a payment that they constrain; Appendix A follows the same groups and gives every statement. The lemmas of Sec. V-C to Sec. V-E that are stated there in prose are printed verbatim in the catalogue: Lemma 1 in Appendix A-D, Lemmas 2 and 3, `Timeout_Race_Blocked` and `Necessity_IMC_Honest_Intermediary_Loses` in Appendix A-H, Lemma 4 in Appendix A-B and Lemma 5 in Appendix A-E; Lemma 6 is printed in Appendix B-F.

F. Skips Require Compromise

The statement of Lemma 6 (Sec. V-E), group 7: every application of `AttackerSkip`, which records `Skipped`, happens after some party's long-term key was compromised. It is proved by `make verify` and re-proved without the liveness restrictions by `make general`.

```
lemma Skipped_Requires_Compromise [reuse]:
  "All p x #i. Skipped(p, x) @ #i ==> Ex P #c. LtkCompromised(P)
      ↪ @ #c & #c < #i"
```

G. PTLC Model: Lock Set-up and Hop Settlement

These are the two rules of the PTLC model (`lightning_aux.spthy`, `-D=PTLC`) that implement the per-hop locks of Sec. VI-E. `Sender_Setup` draws the fresh secret x, the channels `c0` to `c3` and the tweaks t_1, t_2, t_3, derives the channel secrets by the hash chain, locks the sender's channel with $h(s_0)$, gives each hop its incoming channel, outgoing channel, outgoing lock and tweak, and publishes all four locks.

```
rule Sender_Setup:
    let s3 = ~x
        s2 = h(<s3, ~t3>)
        s1 = h(<s2, ~t2>)
        s0 = h(<s1, ~t1>)
    in
    [ Fr(~x), Fr(~c0), Fr(~c1), Fr(~c2), Fr(~c3), Fr(~t1),
      ↪ Fr(~t2), Fr(~t3) ]
    --[ Pay($S, $H1, $H2, $H3, $R, ~c0, ~c1, ~c2, ~c3),
        Neq($S, $H1), Neq($S, $H2), Neq($S, $H3), Neq($S, $R),
        Neq($H1, $H2), Neq($H2, $H3), Neq($H1, $H3),
        Neq($H1, $R), Neq($H2, $R), Neq($H3, $R) ]->
    [ !Lock(~c0, $S, $H1, h(s0)), Open(~c0), SenderWait($S, ~c0),
      !Hop($H1, ~c0, ~c1, $H2, h(s1), ~t1),
      !Hop($H2, ~c1, ~c2, $H3, h(s2), ~t2),
      !Hop($H3, ~c2, ~c3, $R,  h(s3), ~t3),
      !RecvSecret($R, ~c3, ~x),
      Out(h(s0)), Out(h(s1)), Out(h(s2)), Out(h(s3)) ]
```

`Hop_Settle` lets an honest hop that learns the secret `sec` of its outgoing channel claim its incoming channel with the secret derived from `sec` and its own tweak, and publishes that secret.

```
rule Hop_Settle:
    [ Pend($H, cIn, cOut, t), ClaimInfo(cOut, sec), !Lock(cIn,
      ↪ P, $H, Lin), Open(cIn) ]
    --[ Redeemed(cIn), HopSettled($H, cIn, cOut), Eq(h(h(<sec,
      ↪ t>)), Lin) ]->
    [ ClaimInfo(cIn, h(<sec, t>)), Out(h(<sec, t>)) ]
```

In the HTLC baseline (`-D=HTLC`), `Sender_Setup` binds `s0` to `s3` all to `~x`, and `Hop_Settle` passes the outgoing secret on unchanged:

```
rule Hop_Settle:
    [ Pend($H, cIn, cOut, t), ClaimInfo(cOut, sec), !Lock(cIn,
      ↪ P, $H, Lin), Open(cIn) ]
    --[ Redeemed(cIn), HopSettled($H, cIn, cOut), Eq(h(sec),
      ↪ Lin) ]->
    [ ClaimInfo(cIn, sec), Out(sec) ]
```

TABLE IV

LEMMA GROUPS OF `LIGHTNING.SPTHY`; THE CIRCLED NUMBERS IN FIGS. 3 AND 4 REFER TO GROUPS 1 TO 7. A: ALL-TRACES; E: EXISTS-TRACE. UNMARKED ROWS: THE 55 LEMMAS OF `MAKE VERIFY`, ALL VERIFIED (32 ALL-TRACES SAFETY LEMMAS, THE 13 HOP-BYPASS LEMMAS OF GROUP 9, 9 EXISTS-TRACE LEMMAS FOUND AUTOMATICALLY, AND `WORMHOLE_FEE_THEFT_REACHABLE`, REPLAYED FROM A SAVED 116-STEP PROOF). †: PROVED IN `MAKE GENERAL` (LIVENESS RESTRICTIONS REMOVED; SEC. VI). ‡: EXISTS-TRACE, CHECKED WITH T3 REMOVED (TABLE II).

#	Group	Lemmas	Kind
0	Channel layer, keys	`state_update`, `delayed_funds`, `instant_funds`, `settlement_is_traceable`, `No_Punishment_Without_Cheating`, `Funds_Locked_Before_Update`, `Update_Requires_Negotiation`, `Ltk_Known_Implies_Compromised`	A
	non-vacuity	`Protocol_execution`, `Cooperative_Close_Execution`	E
1	Invoice & preimage	`Invoice_Authenticates_Settlement`, `Preimage_Secret_Until_Released`, `Preimage_Never_Known`, `Invoice_Has_Secret_Preimage`, `Invoice_Released_Once`	A
	reachable	`Forged_Invoice_Requires_Key_Compromise`	E
2	Lock causality	`Forward_Requires_Incoming`, `HTLC_On_Opened_Channel`, `Offer_Requires_Invoice`	A
3	Release causality	`Claim_Requires_Release`, `Settle_Requires_Receiver_Release`, `Redeem_Requires_Receiver_Release`, `Fulfilled_Requires_Release`, `Fulfilled_Source`	A
4	Atomicity	`Payment_Atomicity_Under_Liveness`, `Settle_Excludes_Sender_Refund`, `OneOutcomePerHTLC`	A
	non-vacuity	`Multihop_Routing_Reachable`, `Sanity_Honest_Payment_Completes`, `Sanity_Atomic_Payment_Completes`, `Distinct_Parties_Configuration`	E
5	Fees	`Fee_Only_On_Successful_Forward` (per hop, incoming amount = outgoing amount + fee by construction)	A
6	Refund / griefing	`Refund_Requires_Timeout`, `Loss_Requires_Inaction`	A
	reachable	`Refund_Possible`	E
7	Wormhole	`Skipped_Requires_Compromise`	A
	reachable	`Wormhole_Fee_Theft_Reachable` (saved 116-step proof, replayed)	E
8	Intermediary safety, timing	`Intermediary_Never_Loses_Under_Liveness`, `Intermediary_Never_Loses_Strong`, `Timeout_Race_Blocked`, `ForwardTimeGap`	A
	non-vacuity	`Sanity_Compromised_Intermediary_Can_Lose`	E
	necessity ‡	`Necessity_IMC_Honest_Intermediary_Loses`	E
9	Hop bypass	`First_Hop_Skip_Requires_Compromise`, `Second_Hop_Skip_Requires_Compromise`, `Hop3_Skip_Requires_Compromise`, `Hop4_Skip_Requires_Compromise`, `Hop5_Skip_Requires_Compromise`, `Hop6_Skip_Requires_Compromise`	A
	helpers	`Forward_Unique_Per_Incoming`, `SettledHop_From_Forward`, `SettledHop_Uses_Outgoing`, `No_Settle_And_Refund_Same_Hop`, `No_Fulfill_On_Forwarded_Incoming`, `Redeemed_Source`, `SenderSettled_Uses_Fulfilled`	A
	any length †	`Any_Hop_Skip_Requires_Compromise`	A
	helpers †	`Forward_Parent`, `Hop_Redeem_Propagates`, `Sender_Channel_Redeemed`, `Root_First`, `Next_Is_Child`, `Route_Redeemed_Gen`, `Route_Redeemed_No_Compromise`	A